

COMPUTER NETWORKS

Unit 1 — Introduction to Computer Networks & the Internet

Organized Study Notes

Based on: Computer Networking — A Top-Down Approach (Kurose & Ross)

Prepared for MCA Examination Revision

How to use these notes

Each topic is organized as Heading → Definition box → Explanation → Diagram/Formula (where applicable). Every section ends with Self-Check Questions arranged by Bloom's Taxonomy level — Remember, Understand, Apply, Analyze — so you can test recall first and reasoning second.

Table of Contents

1. What Is the Internet?
2. Protocols
3. The Network Edge (Access Networks & Physical Media)
4. The Network Core (Packet Switching & Circuit Switching)
5. Delay, Loss, and Throughput in Packet-Switched Networks
6. Protocol Layers and the Internet Protocol Stack
7. Networks Under Attack — An Introduction to Network Security
8. Quick Revision Sheet — Key Formulas

Each section below follows the same pattern: Heading → Definition box → Explanation → Formula/Table (where applicable) → Self-Check Questions organized by Bloom's Taxonomy level.

1. What Is the Internet?

1.1 Two Ways to Describe the Internet

The Internet can be described in two complementary ways: (1) in terms of its basic hardware and software components — the 'nuts and bolts' view, and (2) in terms of a networking infrastructure that provides services to distributed applications.

DEFINITION The Internet

A worldwide computer network that interconnects billions of computing devices (hosts / end systems) throughout the world, connected together by a network of communication links and packet switches.

1.1.1 Nuts-and-Bolts Description

- End systems / hosts: desktops, laptops, smartphones, servers, IoT devices — devices that run network applications.
- Communication links: made of different physical media — coaxial cable, copper wire, fiber optics, radio spectrum.
- Packet switches: forward incoming data onto an outgoing link. Two main types: routers (used in the network core) and link-layer switches (used in access networks).
- Route / Path: the sequence of communication links and packet switches traversed by a packet from sending host to receiving host.
- ISPs (Internet Service Providers): residential ISPs (cable/telephone companies), corporate ISPs, university ISPs, WiFi-access ISPs, and cellular data ISPs. Each ISP is itself a network of packet switches and links.
- TCP/IP: the Internet's principal protocols — Transmission Control Protocol (TCP) and Internet Protocol (IP). IP specifies the format of packets sent between routers and end systems.
- Internet standards are developed by the IETF (Internet Engineering Task Force); the standards documents are called RFCs (Requests for Comments). There are close to 9,000 RFCs, defining protocols such as TCP, IP, HTTP, and SMTP.

1.1.2 Services Description

Viewed as infrastructure for applications, the Internet allows distributed applications running on end systems to exchange data. End systems attached to the Internet provide a socket interface that specifies how a program on one end system asks the Internet infrastructure to deliver data to a specific destination program on another end system.

DEFINITION Socket Interface

A set of rules that a sending program must follow so that the Internet infrastructure can deliver data to the specified destination program on another end system — analogous to the rules a postal system requires for delivering a letter.

1.1.3 Hosts, Clients, and Servers

- End systems are also called hosts because they host (run) application programs, e.g., a Web browser.

- Hosts are divided into clients and servers: clients are typically desktops, laptops and smartphones; servers are more powerful machines that store and distribute Web pages, stream video, and relay e-mail.

Self-Check Questions — What Is the Internet

■ REMEMBER *(recall facts & definitions)*

- 1. Define 'the Internet' in one sentence.
- 2. What do the acronyms IETF and RFC stand for?
- 3. List the two most important protocols that make up TCP/IP.

■ UNDERSTAND *(explain in your own words)*

- 1. Explain the difference between the 'nuts-and-bolts' description and the 'services' description of the Internet.
- 2. In your own words, describe what a socket interface does and why it is needed.

■ APPLY *(solve / calculate)*

- 1. Given the two descriptions of the Internet, classify the following as either hardware/nuts-and-bolts or service-oriented: (a) a fiber-optic cable, (b) a video-conferencing app exchanging data between two hosts.

■ ANALYZE *(compare, differentiate, break down)*

- 1. Compare a client and a server in terms of role and typical hardware capability.
- 2. Differentiate between a router and a link-layer switch in terms of where each is typically deployed.

2. Protocols

DEFINITION Protocol

A protocol defines the format and the order of messages exchanged between two or more communicating entities, as well as the actions taken on the transmission and/or receipt of a message or other event.

All activity in the Internet that involves two or more communicating remote entities is governed by a protocol. Human protocols exist too — for example, the exchange of greetings when two people meet is a kind of human 'protocol' that dictates when and what a participant is expected to communicate.

Why protocols matter

- They ensure that all devices/software 'speak the same language' at each step of communication.
- They specify message format (syntax), message order (sequencing), and the actions to be taken on sending/receiving a message or on timeout.

Self-Check Questions — Protocols

■ REMEMBER (recall facts & definitions)

- 1. State the textbook definition of a 'protocol'.

■ UNDERSTAND (explain in your own words)

- 1. Using a human analogy (e.g., a phone call or greeting), explain what a protocol is and why both parties must follow it.

■ APPLY (solve / calculate)

- 1. Identify three elements that must be specified when you design a new communication protocol.

■ ANALYZE (compare, differentiate, break down)

- 1. Analyze why a mismatch in protocol between a client and a server would cause communication to fail, even if the physical link is working correctly.

3. The Network Edge

3.1 End Systems and Access Networks

End systems access the Internet through Internet Service Providers (ISPs). Access networks are the links that physically connect an end system to the first router (the 'edge router') on a path to any other distant end system.

3.2 Types of Broadband Residential Access

DSL (Digital Subscriber Line): typically provided by the same local telephone company (telco) that provides wired local phone access, so a residence's telco is also its ISP for DSL.

Cable: uses the cable television infrastructure; the cable company is the ISP.

3.3 Physical Media

For a transmitter-receiver pair, a bit is sent by propagating electromagnetic waves or optical pulses across a physical medium. Physical media fall into two broad categories:

DEFINITION Guided Media

Waves are guided along a solid medium such as a fiber-optic cable, twisted-pair copper wire, or coaxial cable.

DEFINITION Unguided Media

Waves propagate freely in the atmosphere or in outer space, e.g., in a wireless LAN or a digital satellite channel.

3.3.1 Twisted-Pair Copper Wire

- The least expensive and most commonly used guided transmission medium; used by telephone networks for over a hundred years.
- More than 99% of wired connections from a telephone handset to the local switch use twisted-pair copper wire.
- Consists of two insulated copper wires (~1 mm thick) arranged in a regular spiral pattern; twisting reduces electrical interference from nearby pairs.
- Unshielded Twisted Pair (UTP) is commonly used for LANs within a building; data rates range from 10 Mbps to 10 Gbps.
- Category 6a cable can achieve 10 Gbps for distances up to about 100 meters. Twisted pair is now the dominant solution for high-speed LAN networking.

3.3.2 Coaxial Cable

- Consists of two copper conductors, but concentric rather than parallel (unlike twisted pair).
- Special insulation and shielding allow high data transmission rates; widely used in cable television systems.
- Can be used as a guided shared medium — multiple end systems connect directly to the cable, and each end system receives whatever is sent by the others.

3.3.3 Fiber Optics

- A thin, flexible medium that conducts pulses of light, with each pulse representing a bit.
- Supports very high bit rates — up to tens or even hundreds of gigabits per second.
- Immune to electromagnetic interference; very low signal attenuation up to 100 km.
- Optical Carrier (OC) standard link speeds range from 51.8 Mbps to 39.8 Gbps, referred to as OC-n, where link speed = $n \times 51.8$ Mbps.

3.3.4 Terrestrial Radio Channels

- Require no physical wire, can penetrate walls, support mobility, and can carry signals over long distances.
- Affected by: path loss & shadow fading (signal weakens with distance/obstacles), multipath fading (reflection off objects), and interference.
- Classified into three groups by range: very short distance (1–2 m, e.g., wireless headsets/keyboards), local area (10 m to a few hundred meters), and wide area (tens of kilometers).

3.3.5 Satellite Radio Channels

- A communication satellite links two or more Earth-based microwave transmitter/receivers called ground stations.
- The satellite receives on one frequency band, regenerates the signal with a repeater, and retransmits on another frequency.

DEFINITION Geostationary satellites

Remain permanently above the same spot on Earth by orbiting at ~36,000 km above the Earth's surface.

DEFINITION LEO (Low-Earth Orbiting) satellites

Placed much closer to Earth, do not stay above a fixed spot, rotate around the Earth, and may communicate with each other and ground stations; many satellites are required for continuous area coverage.

Self-Check Questions — The Network Edge & Physical Media

REMEMBER (recall facts & definitions)

- 1. Define guided media and unguided media.
- 2. Name the two types of broadband residential access described.
- 3. What is a geostationary satellite?

UNDERSTAND (explain in your own words)

- 1. Explain why twisted-pair copper wire is still the dominant medium for high-speed LAN networking despite being the least expensive option.
- 2. Describe, in your own words, the difference between path loss, multipath fading, and interference for a terrestrial radio channel.

APPLY (solve / calculate)

- 1. If an OC-48 link is used, calculate its approximate link speed ($n \times 51.8$ Mbps).

- 2. For a wireless network, classify each of the following as short-distance, local-area, or wide-area: a Bluetooth headset, a home WiFi router, a cellular tower link.

■ **ANALYZE** (*compare, differentiate, break down*)

- 1. Compare twisted-pair, coaxial cable, and fiber optics on cost, data rate, and typical use case.
- 2. Differentiate between geostationary and LEO satellites in terms of altitude, coverage, and delay implications.

4. The Network Core

4.1 Packet Switching

When one end system has data to send to another, the sending end system segments the data and adds header bytes to each segment, forming packets. A packet switch takes a packet arriving on one of its incoming links and forwards it on one of its outgoing links. Link-layer switches are typically used in access networks, while routers are typically used in the network core.

DEFINITION Packet

A package of information — data segmented by the sending end system with header bytes added — sent through the network to the destination end system, where the original data is reassembled.

4.1.1 Store-and-Forward Transmission

DEFINITION Store-and-Forward Transmission

The packet switch must receive the entire packet before it can begin to transmit the first bit of that packet onto the outbound link.

For a packet of L bits sent over a link with transmission rate R bits/sec, the time to transmit the packet is L / R seconds. For a path of N links each of rate R , there are $N - 1$ routers between source and destination, and (ignoring propagation/queuing delay) the end-to-end store-and-forward delay is $N \times L / R$.

4.1.2 Queuing Delay and Packet Loss

Each packet switch has multiple attached links, and for each link it maintains an output buffer (output queue) that stores packets about to be sent on that link. If an arriving packet finds the link busy with another transmission, it must wait in the output buffer — this is the queuing delay, which is variable and depends on network congestion.

- Because a real queue has finite capacity, an arriving packet may find a full queue — the router then drops the packet, resulting in packet loss.

4.2 Circuit Switching

In circuit-switched networks, the resources needed along a path (buffers, link transmission rate) are reserved for the entire duration of the communication session. In packet-switched networks, resources are not reserved — a session's packets use resources on demand and may have to queue for access to a link.

4.2.1 Multiplexing in Circuit-Switched Networks

A circuit in a link is implemented with either FDM or TDM:

DEFINITION Frequency-Division Multiplexing (FDM)

The frequency spectrum of a link is divided among the connections established across the link; each connection is dedicated a frequency band for its entire duration (e.g., a telephone frequency band is typically 4 kHz wide; FM radio stations share 88–108 MHz using FDM).

DEFINITION Time-Division Multiplexing (TDM)

Time is divided into frames of fixed duration, and each frame is divided into a fixed number of time slots. The network dedicates one time slot in every frame to each connection. The transmission rate of a circuit equals the frame rate multiplied by the number of bits in a slot.

4.3 Packet Switching vs. Circuit Switching

Aspect	Circuit Switching	Packet Switching
Resource reservation	Reserved for the entire session duration	Not reserved; used on demand
Delay behaviour	Predictable, guaranteed once connection set up	Variable — packets may queue at switches
Efficiency with bursty data	Wastes idle-circuit capacity	Efficient — statistical multiplexing
Failure mode under congestion	New connections blocked (busy signal)	Increased delay / possible packet loss
Multiplexing technique	FDM or TDM	Statistical multiplexing of packets

Self-Check Questions — The Network Core

REMEMBER (recall facts & definitions)

- 1. Define store-and-forward transmission.
- 2. What is queuing delay?
- 3. State the formula for transmission time of a packet of L bits over a link of rate R .

UNDERSTAND (explain in your own words)

- 1. Explain why packet loss occurs even though, in principle, we could imagine a queue of infinite capacity.
- 2. In your own words, describe the difference between how circuit switching and packet switching allocate network resources.

APPLY (solve / calculate)

- 1. Calculate the transmission delay for a 1,500-byte (12,000-bit) packet sent over a 10 Mbps link.
- 2. A path has $N = 4$ links, each of rate $R = 100$ Mbps, and packet length $L = 8,000$ bits. Ignoring propagation and queuing delay, compute the total store-and-forward delay ($N \times L/R$).
- 3. A TDM link has frames with 4 time slots of 1,000 bits each, and the frame rate is 8,000 frames/sec. Find the transmission rate of one circuit.

ANALYZE (compare, differentiate, break down)

- 1. Compare FDM and TDM in terms of how they divide link resources among connections.
- 2. Analyze why packet switching is generally preferred over circuit switching for bursty data traffic such as Web browsing.

5. Delay, Loss, and Throughput in Packet-Switched Networks

A packet starts in a host (source), passes through a series of routers, and ends its journey in another host (destination). At each node along the path, the packet suffers several types of delay. The four principal delay components — nodal processing delay, queuing delay, transmission delay, and propagation delay — accumulate to give the total nodal delay.

5.1 The Four Types of Delay

5.1.1 Processing Delay

DEFINITION Processing Delay

The time required to examine a packet's header, determine where to direct it, and check for bit-level errors that occurred while transmitting the packet's bits from the upstream node.

5.1.2 Queuing Delay

DEFINITION Queuing Delay

The time a packet waits in the output queue to be transmitted onto the link, depending on the number of earlier-arriving packets already queued. It is zero if the queue is empty, and large if traffic is heavy.

5.1.3 Transmission Delay

DEFINITION Transmission Delay

The time required to push (transmit) all of a packet's L bits into the link, equal to L/R , where R is the link's transmission rate. It depends on packet length and link rate — not on the distance between routers.

5.1.4 Propagation Delay

DEFINITION Propagation Delay

The time required for a bit to propagate from one router to the next, equal to d/s , where d is the distance between the two routers and s is the propagation speed of the link (roughly 2×10^8 to 3×10^8 m/s). It depends on distance — not on packet length or transmission rate.

Key distinction

Transmission delay is the time to push all the packet's bits onto the link (function of packet length L and rate R). Propagation delay is the time for a bit to travel across the link (function of distance d and propagation speed s). Students commonly confuse these two — remember: transmission = 'how long to shove the packet out', propagation = 'how long it travels once out.'

5.2 Traceroute

DEFINITION Traceroute

A diagnostic program available on any Internet host: the source sends multiple special packets toward a specified destination hostname; as they travel toward the destination through a series of routers, each router that receives one of these packets sends a short message back to the source containing that router's name and address.

5.3 Packet Loss

A queue preceding a link has finite capacity. Because of this, packet delays do not approach infinity even as traffic intensity approaches 1 — instead, an arriving packet may find a full queue. With no place to store it, the router drops (loses) the packet.

5.4 Throughput

DEFINITION Throughput

The rate (in bits/sec) at which bits are being transferred between sender and receiver. Instantaneous throughput is the rate at a given instant; average throughput of a file transfer of F bits taking T seconds is F/T bits/sec.

Self-Check Questions — Delay, Loss and Throughput

REMEMBER (recall facts & definitions)

- 1. List the four components of total nodal delay.
- 2. Give the formula for average throughput of a file transfer.
- 3. Define packet loss.

UNDERSTAND (explain in your own words)

- 1. Explain, in your own words, why transmission delay does not depend on the distance between two routers, while propagation delay does not depend on packet length.
- 2. Describe how traceroute is able to report the routers along a path.

APPLY (solve / calculate)

- 1. A 2,000-bit packet is sent over a 2 Mbps link across a distance of 1,000 km, with a propagation speed of 2×10^8 m/s. Calculate the transmission delay and the propagation delay separately.
- 2. A file of 4,000,000 bits is transferred in 8 seconds. Calculate the average throughput.

ANALYZE (compare, differentiate, break down)

- 1. Analyze what happens to queuing delay as traffic intensity approaches 1, assuming (a) infinite queue capacity and (b) finite queue capacity.
- 2. Compare processing delay and queuing delay in terms of what causes each and how predictable each one is.

6. Protocol Layers and the Internet Protocol Stack

The Internet protocol stack consists of five layers: the physical, link, network, transport, and application layers. Each layer offers services to the layer above it by performing certain actions and by using the services of the layer directly below it.

6.1 Application Layer

DEFINITION Application Layer

The layer where network applications and their application-layer protocols reside. Includes protocols such as HTTP (Web document request/transfer), SMTP (e-mail transfer), and FTP (file transfer between two end systems).

An application-layer protocol is distributed over multiple end systems, with the application in one end system using the protocol to exchange packets of information with the application in another end system.

6.2 Transport Layer

DEFINITION Transport Layer

Transports application-layer messages between application endpoints. The Internet has two transport protocols, TCP and UDP, either of which can transport application-layer messages.

- TCP: connection-oriented service; provides guaranteed delivery of application-layer messages, flow control (sender/receiver speed matching), breaks long messages into shorter segments, and provides congestion control.
- UDP: connectionless service — a 'no-frills' service providing no reliability, no flow control, and no congestion control.

6.3 Network Layer

DEFINITION Network Layer

Responsible for moving network-layer packets, known as datagrams, from one host to another. The source's transport-layer protocol (TCP or UDP) passes a segment and a destination address to the network layer, which delivers the segment to the transport layer at the destination host. The network layer also contains routing protocols that determine routes between source and destination.

6.4 Link Layer

DEFINITION Link Layer

Moves a datagram from one node (host or router) to the next node along a route. At each node, the network layer passes the datagram down to the link layer, which delivers it to the link layer at the next node, which passes it back up to the network layer. Examples: Ethernet, WiFi, and the cable access network's DOCSIS protocol.

6.5 Physical Layer

DEFINITION Physical Layer

While the link layer's job is to move entire frames from one network element to an adjacent one, the physical layer's job is to move the individual bits within the frame from one node to the next, over a physical medium such as twisted-pair copper wire or single-mode fiber optics.

6.6 The Five Layers at a Glance

Layer	Unit of Data	Key Protocols / Examples	Core Responsibility
Application	Message	HTTP, SMTP, FTP	Supports network applications
Transport	Segment	TCP, UDP	Process-to-process delivery of messages
Network	Datagram	IP, routing protocols	Host-to-host routing of datagrams
Link	Frame	Ethernet, WiFi, DOCSIS	Node-to-node delivery over one link
Physical	Bit	Twisted pair, fiber optics	Moving individual bits across the medium

Self-Check Questions — Protocol Layers

REMEMBER (recall facts & definitions)

- 1. Name the five layers of the Internet protocol stack, from top to bottom.
- 2. Which two protocols operate at the transport layer?

UNDERSTAND (explain in your own words)

- 1. Explain the difference in services offered by TCP versus UDP at the transport layer.
- 2. Describe, in your own words, how a datagram travels from the network layer down to the link layer and back up at each node along a route.

APPLY (solve / calculate)

- 1. For a Web browsing request, identify which protocol from each of the five layers would typically be involved.

ANALYZE (compare, differentiate, break down)

- 1. Compare the network layer and the link layer in terms of scope (end-to-end vs. node-to-node) and the unit of data each works with.

- 2. Analyze why an application that cannot tolerate any delay in error recovery (e.g., live video) might prefer UDP over TCP despite UDP's lack of reliability.

7. Networks Under Attack — An Introduction to Network Security

The Internet was originally designed with relatively little thought given to security. As it has grown into a critical infrastructure, it has become a prime target for attackers. This section introduces some common classes of attack.

7.1 Denial-of-Service (DoS) Attacks

DEFINITION Denial-of-Service (DoS) Attack

An attack that renders a network, host, or other piece of infrastructure unusable by legitimate users, by overwhelming a targeted resource with bogus traffic or requests.

Most Internet DoS attacks fall into one of three categories:

1. **Vulnerability attack:** sending a few well-crafted messages to a vulnerable application or operating system running on a targeted host. If the right sequence of packets is sent, the service can stop, or the host can crash.
2. **Bandwidth flooding:** the attacker sends a deluge of packets to the targeted host — so many that the target's access link becomes clogged, preventing legitimate packets from reaching the server.
3. **Connection flooding:** the attacker establishes a large number of half-open or fully open TCP connections at the target host, bogging it down so it stops accepting legitimate connections.

7.2 Distributed Denial-of-Service (DDoS)

DEFINITION Distributed DoS (DDoS) Attack

The attacker controls multiple compromised sources (often a botnet with thousands of hosts) and has each source blast traffic at the target simultaneously. The aggregate traffic rate across all controlled sources needs to be approximately R to cripple the service. DDoS attacks are much harder to detect and defend against than a single-source DoS attack.

7.3 Packet Sniffing

DEFINITION Packet Sniffer

A passive receiver that records a copy of every packet that flies by it on a shared broadcast medium, without injecting any packets into the channel.

Packets can contain all kinds of sensitive information, including passwords, social security numbers, trade secrets, and private personal messages — making packet sniffing a serious privacy and security threat, especially on shared/wireless media.

Self-Check Questions — Network Security

■ **REMEMBER** (recall facts & definitions)

- 1. Define a Denial-of-Service (DoS) attack.
- 2. What makes an attack a 'distributed' DoS (DDoS) attack rather than a plain DoS attack?

- 3. What is a packet sniffer?

■ **UNDERSTAND** (*explain in your own words*)

- 1. Explain, in your own words, the difference between a vulnerability attack, a bandwidth-flooding attack, and a connection-flooding attack.
- 2. Describe why a packet sniffer is described as 'passive.'

■ **APPLY** (*solve / calculate*)

- 1. Given a scenario where an attacker opens thousands of half-open TCP connections to a server, identify which category of DoS attack this represents.

■ **ANALYZE** (*compare, differentiate, break down*)

- 1. Analyze why DDoS attacks are harder to detect and defend against compared to single-source DoS attacks.
- 2. Compare a bandwidth-flooding attack with a connection-flooding attack in terms of what network resource each exhausts.

8. Quick Revision Sheet — Key Formulas

Quantity	Formula	Notes
Transmission delay	$d_{\text{trans}} = L / R$	L = packet length (bits), R = link rate (bits/sec)
Propagation delay	$d_{\text{prop}} = d / s$	d = distance, s = propagation speed (~ $2-3 \times 10^8$ m/s)
Total nodal delay	$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$	Sum of all four delay components
End-to-end store-and-forward delay (no queuing)	$d_{\text{end-to-end}} = N \times L / R$	N links, each of rate R
Average throughput	Throughput = F / T	F = file size in bits, T = transfer time in seconds
OC-n link speed	Speed = $n \times 51.8$ Mbps	Optical Carrier standard
TDM circuit rate	Rate = frame rate $\times$ bits per slot	For one dedicated slot per frame

End of Unit 1 Notes — Good luck with your exam!

COMPUTER NETWORKS

MCA Examination Study Notes

UNIT: THE APPLICATION LAYER

Client-Server & P2P Architectures • HTTP & the Web • Electronic Mail • DNS • P2P File Distribution • Video Streaming & CDNs • Socket Programming

Compiled & Restructured From

the learner's own lecture notes on **Computer Networking: A Top-Down Approach**
(Kurose & Ross)

Reorganised with headings, formal definitions, comparison tables,
worked examples, and a Bloom's-Taxonomy-based question bank
for exam revision.

How to Use These Notes

These notes have been reorganised from the original chapter-wise material into a single, exam-ready study unit. Every topic follows the same predictable pattern so that revision is fast: a short heading, a boxed formal definition, a plain-language explanation, a worked example, and — wherever it helps — a comparison table or a simple diagram.

Why Bloom's Taxonomy?

Bloom's Taxonomy is a six-level model of learning, ordered from the simplest cognitive skill to the most complex: Remember → Understand → Apply → Analyze → Evaluate → Create. Most students stop at the first two levels (memorising definitions and understanding explanations) and are then surprised when the exam asks them to compare two protocols, justify a design choice, or design a solution. This document is deliberately built to push you through all six levels for every important topic.

Level	What it asks of you	How it appears in these notes
1. Remember	Recall a fact, term, or definition	The blue "DEFINITION" boxes
2. Understand	Explain the idea in your own words	The explanatory paragraphs after each definition
3. Apply	Use the concept in a concrete situation	"Example" / "worked scenario" text and diagrams
4. Analyze	Break an idea down / compare two things	Comparison tables (e.g. TCP vs UDP)
5. Evaluate	Judge, justify, or critique a choice	"Evaluate" questions in the question bank
6. Create	Design or propose something new	"Create" questions in the question bank

Key to the Boxes Used Throughout

DEFINITION — Sample Term

A short, exam-quotable definition appears here — this is what you should be able to reproduce from memory.

KEY POINT

A key point highlights something examiners love to test, or a distinction that is easy to get wrong.

EXAM TIP

An exam tip tells you how a topic is typically asked (2-mark / 5-mark / 10-mark) or gives a mnemonic.

COMMON CONFUSION

A caution box flags two concepts that students routinely mix up in exams.

Suggested Revision Plan

- First pass: Read every DEFINITION box and KEY POINT box only, chapter by chapter (Remember + Understand).
- Second pass: Study every comparison table and diagram until you can redraw them from memory (Analyze).
- Third pass: Attempt the Bloom's Question Bank (Chapter 9) without looking back — check answers after.
- Night before exam: Skim Chapter 8 (Quick Revision Tables) and Chapter 10 (Important Questions) only.

Contents

1. Principles of Network Applications

- 1.1 Network Application Architectures (Client-Server & P2P)
- 1.2 Processes Communicating (Sockets, Addressing)
- 1.3 Transport Services Available to Applications
- 1.4 Transport Services Provided by the Internet (TCP & UDP)
- 1.5 Application-Layer Protocols

2. The Web and HTTP

- 2.1–2.2 Overview, Non-Persistent & Persistent Connections
- 2.3 HTTP Message Format
- 2.4 Cookies
- 2.5 Web Caching
- 2.6 The Conditional GET
- 2.7 HTTP/2 and Framing

3. Electronic Mail in the Internet (SMTP, Mail Formats, POP3/IMAP)

4. DNS — The Internet's Directory Service

5. Peer-to-Peer (P2P) File Distribution & BitTorrent

6. Video Streaming and Content Distribution Networks

7. Socket Programming (UDP and TCP)

8. Quick Revision — Master Summary Tables

9. Bloom's Taxonomy Question Bank (Remember → Create)

10. Important Questions — Exam Checklist (2 / 5 / 10 Marks)

1. Principles of Network Applications

Network applications are the reason computer networks exist — the Web, e-mail, file sharing, video calling and streaming are all network applications. This chapter builds the vocabulary needed for every later topic: how an application is structured across end systems, how its processes talk to each other, and what an application can expect from the transport layer underneath it.

1.1 Network Application Architectures

The application architecture is designed by the application developer and dictates how the application is structured over the various end systems. It is a design-level decision, distinct from the network architecture (which is fixed by the underlying network, e.g. the five-layer Internet model). A developer usually chooses between two dominant paradigms: client-server and peer-to-peer (P2P).

1.1.1 Client-Server Architecture

DEFINITION — Client-Server Architecture

An architecture in which an always-on host, called the server, services requests from many other hosts, called clients; clients do not communicate directly with one another.

The server has a fixed, well-known IP address, and because it is always on, a client can always reach it by sending a packet to that address. A classic example is the Web: an always-on Web server responds to requests from browsers running on client hosts. Well-known client-server applications include the Web, FTP, Telnet and e-mail. A single server is often insufficient for a popular service, so companies build large data centres containing many servers that together create a powerful virtual server.

Client-Server Model

```
Client A  ----request---->
                        [  SERVER  ]
Client A  <---response---- (always on,
                           fixed IP address)
Client B  ----request---->
Client B  <---response----

Clients never talk directly to each other.
```

1.1.2 Peer-to-Peer (P2P) Architecture

DEFINITION — P2P Architecture

An architecture with minimal (or no) reliance on dedicated servers; instead, pairs of intermittently connected hosts called peers communicate directly with each other.

Peers are not owned by the service provider — they are ordinary desktops and laptops controlled by users, typically located in homes, universities and offices. BitTorrent (file sharing) is a popular example. The most compelling feature of P2P is self-scalability: each peer both consumes and supplies service capacity, so as more peers arrive, more capacity is added to the system automatically. P2P is also cost-effective because it needs little dedicated server infrastructure, but it faces real challenges of security, performance and reliability because of its highly decentralised structure.

Aspect	Client-Server	Peer-to-Peer (P2P)
Server dependency	Always-on dedicated server required	Minimal or no dedicated server
Addressing	Server has a fixed, well-known IP address	Peers connect/disconnect intermittently
Scalability	Must scale server/data-centre capacity manually	Self-scalable — peers add capacity as they join
Cost	High (server hardware + bandwidth)	Low — no major server infrastructure
Security / control	Easier to secure and manage centrally	Harder — decentralised, less control
Examples	Web, FTP, Telnet, e-mail	BitTorrent and other file-sharing apps

EXAM TIP

“Compare client-server and P2P architectures” is an extremely common 5-mark question — memorise the table above.

1.2 Processes Communicating

DEFINITION — Process

A program that is running within an end system.

Processes on the same end system communicate using inter-process communication, governed by the operating system. Processes on two different end systems communicate by exchanging messages across the computer network: a sending process creates and sends messages into the network, and a receiving process receives them and possibly responds.

1.2.1 Client and Server Processes

A network application consists of pairs of processes that send messages to each other. For any communicating pair, one process is labelled the client and the other the server. In the Web, the browser process is the client and the Web-server process is the server. In P2P file sharing, the peer downloading the file is the client and the peer uploading it is the server. More generally, the process that initiates the communication (contacts the other process first) is the client; the process that waits to be contacted is the server.

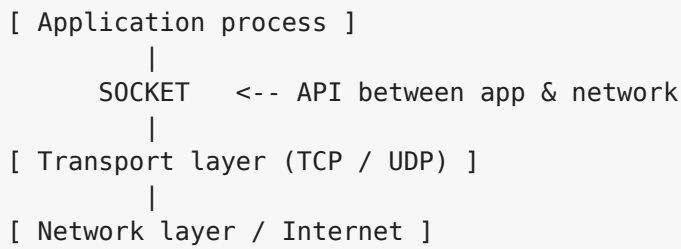
1.2.2 Sockets — The Interface Between Process and Network

DEFINITION — Socket

The software interface between the application layer and the transport layer within a host; a process sends messages into, and receives messages from, the network through its socket.

A socket is also called the Application Programming Interface (API) between the application and the network, because it is the programming interface with which network applications are built. As a developer, you have control on the application-layer side of the socket but almost no control over the transport-layer side beyond choosing the transport protocol and a handful of parameters.

The Socket as an Interface



1.2.3 Addressing Processes

To send a message to a process, two pieces of information must be specified: (1) the address of the host, and (2) an identifier that specifies the particular receiving process at that host.

- Host address: in the Internet this is the IP address, a 32-bit quantity that uniquely identifies the host.
- Process identifier: a destination port number. Well-known applications are assigned specific port numbers — e.g. a Web server (HTTP) uses port 80, and a mail server (SMTP) uses port 25.

KEY POINT

A complete “address” for a process is the pair (IP address, port number) — the IP address gets the packet to the right host, and the port number gets it to the right process/socket on that host.

1.3 Transport Services Available to Applications

The services that a transport-layer protocol can offer an application can be broadly classified along four dimensions: reliable data transfer, throughput, timing, and security.

1.3.1 Reliable Data Transfer

DEFINITION — Reliable Data Transfer

A guarantee, provided by a transport protocol, that data sent by one end of an application is delivered correctly and completely to the other end.

Packets can be lost inside a network — a router buffer can overflow, or a host/router can discard a packet whose bits are corrupted. For applications such as e-mail, file transfer, remote host access, Web transfers and financial applications, data loss can have devastating consequences, so these applications need reliable data transfer. Loss-tolerant applications, most notably conversational audio/video, can tolerate some data loss — a lost packet just causes a brief glitch, not a crucial impairment.

1.3.2 Throughput

DEFINITION — Available Throughput

The rate at which a sending process can deliver bits to a receiving process, in the context of a communication session along a network path.

Because other sessions share the same links and come and go over time, available throughput fluctuates. Bandwidth-sensitive applications have a specific throughput requirement (e.g. Internet telephony), while elastic applications (e-mail, file transfer, Web transfers) can make use of as much or as little throughput as happens to be available — more is always better, but they still function at low throughput.

1.3.3 Timing

A transport-layer protocol can also provide timing guarantees, for example ensuring that every bit the sender pumps into the socket arrives at the receiver within a bounded delay (say, 100 msec). This matters for interactive real-time applications such as Internet telephony, virtual environments, teleconferencing and multiplayer games, all of which need tight timing constraints to feel responsive.

1.3.4 Security

A transport protocol can provide security services such as encrypting data at the sender and decrypting it at the receiver (confidentiality), as well as data integrity and end-point authentication, even if the data is intercepted in transit.

Application	Data Loss Tolerance	Throughput Need	Time Sensitivity
File transfer / download	No loss	Elastic	No
E-mail	No loss	Elastic	No
Web documents	No loss	Elastic (few kbps)	No
Internet telephony / video conferencing	Loss-tolerant	Audio: few kbps–1 Mbps; Video: 10 kbps–5 Mbps	Yes: 100s of msec
Streaming stored audio/video	Loss-tolerant	Same as above	Yes: few seconds
Interactive games	Loss-tolerant	Few kbps–10 kbps	Yes: 100s of msec
Smartphone messaging	No loss	Elastic	Yes and no

1.4 Transport Services Provided by the Internet

TCP/IP networks — and the Internet in particular — make two transport protocols available to applications: TCP and UDP. A developer creating a new application must choose one of the two.

1.4.1 TCP Services

When an application invokes TCP as its transport protocol, it receives two core services:

- **Connection-oriented service:** TCP has the client and server exchange transport-layer control information before application messages begin to flow (“handshaking”). This prepares both ends for the onslaught of packets. After handshaking, a full-duplex TCP connection exists between the two sockets, so both processes can send messages simultaneously.
- **Reliable data transfer service:** The communicating processes can rely on TCP to deliver all data sent, without error and in the proper order — no missing or duplicate bytes.

TCP also includes a congestion-control mechanism, which throttles a sending process when the network is congested — this benefits the general welfare of the Internet rather than the two communicating processes directly.

1.4.2 UDP Services

UDP is a no-frills, lightweight, connectionless transport protocol providing minimal services:

- No handshaking before the two processes start to communicate.

- Unreliable data transfer: no guarantee that a sent message will ever reach the receiving process, and messages that do arrive may arrive out of order.
- No congestion control: the sender can pump data into the network layer at any rate it pleases (though actual end-to-end throughput may still be capped by link capacity or congestion).

Feature	TCP	UDP
Connection	Connection-oriented (handshake first)	Connectionless
Reliability	Reliable — guarantees in-order, error-free delivery	Unreliable — no delivery or order guarantee
Congestion control	Yes — throttles sender when network is congested	No — sends at any rate
Speed / overhead	Slower, more overhead (acks, retransmits)	Faster, minimal overhead
Typical uses	Web (HTTP), e-mail (SMTP), file transfer (FTP)	DNS, streaming media, Internet telephony

COMMON CONFUSION

Neither TCP nor UDP provides timing or minimum-throughput guarantees, and TCP does not provide security by itself (that is why HTTPS layers TLS/SSL on top of TCP).

1.4.3 Application → Protocol Mapping

Application	Application-Layer Protocol	Underlying Transport Protocol
Electronic mail	SMTP [RFC 5321]	TCP
Remote terminal access	Telnet [RFC 854]	TCP
Web	HTTP 1.1 [RFC 7230]	TCP
File transfer	FTP [RFC 959]	TCP
Streaming multimedia	HTTP (e.g. YouTube), DASH	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary (e.g. Skype)	UDP or TCP

KEY POINT

Internet telephony applications are often designed to fall back to TCP if UDP communication fails, even though UDP is the natural first choice for real-time media.

1.5 Application-Layer Protocols

DEFINITION — Application-Layer Protocol

A protocol that defines how an application's processes, running on different end systems, pass messages to

each other.

An application-layer protocol defines:

- The types of messages exchanged — e.g. request messages and response messages.
- The syntax of each message type — the fields in the message and how they are delineated.
- The semantics of the fields — the meaning of the information carried in each field.
- Rules that determine when and how a process sends messages, and how it responds to messages.



COMMON CONFUSION

An application and an application-layer protocol are not the same thing. HTTP is an application-layer protocol that is only one piece of the Web application, which also needs standards for object formatting (e.g. HTML) and other pieces to work.

2. The Web and HTTP

The Web was the application that made the Internet a household name. It runs on the HyperText Transfer Protocol (HTTP), defined in RFC 1945, RFC 7230 and RFC 7540. HTTP is implemented in two programs — a client program and a server program — running on different end systems and talking to each other by exchanging HTTP messages.

2.1 Overview of HTTP

DEFINITION — HTTP (HyperText Transfer Protocol)

The Web's application-layer protocol, which defines the structure of request and response messages exchanged between a browser (client) and a Web server.

A Web page (or document) consists of objects — simply files such as an HTML file, a JPEG image, a JavaScript file, a CSS style-sheet file, or a video clip, each addressable by a single URL. Most Web pages consist of a base HTML file plus several referenced objects. HTTP uses TCP, not UDP, as its underlying transport protocol. The client first initiates a TCP connection with the server; once established, the browser and server access TCP through their socket interfaces. The client sends HTTP request messages into its socket and receives HTTP response messages from it; the server does the mirror image.

KEY POINT

HTTP itself is described as a 'stateless protocol' — the server maintains no information about past client requests by default. This is precisely why cookies had to be invented (see 2.4).

2.2 Non-Persistent and Persistent Connections

2.2.1 Non-Persistent Connections

DEFINITION — Non-Persistent Connection

A connection in which each TCP connection is closed after the server sends the requested object — the connection does not persist for other objects. HTTP/1.0 uses non-persistent connections.

Steps for One Object over a Non-Persistent Connection

1. Client initiates a TCP connection to server (port 80).
2. Client sends an HTTP request message via its socket.
3. Server retrieves the object, sends an HTTP response, then tells TCP to close the connection.
4. Client receives the response; connection terminates.
5. Repeat steps 1-4 for EACH referenced object (e.g. each of 10 embedded JPEG images).

Each non-persistent TCP connection transports exactly one request message and one response message. The round-trip time (RTT) is the time for a small packet to travel from client to server and back, including propagation, queuing and processing delays. When a user clicks a hyperlink, the browser must first perform a three-way TCP handshake (small segment out, ack + segment back, final ack) before it can even send the HTTP request — so the total response time for one object is roughly 2 RTTs plus the server's transmission time for the file.

COMMON CONFUSION

A common shortcoming of non-persistent connections: a brand-new TCP connection must be established and maintained for every single object, and TCP buffers/variables must be allocated at both client and server each time — wasteful when a page has many embedded objects.

2.2.2 Persistent Connections

DEFINITION — Persistent Connection

A connection in which the server leaves the TCP connection open after sending a response, so that subsequent requests and responses between the same client and server can be sent over the same connection. Used by default in HTTP/1.1.

With persistent connections, an entire Web page — the base HTML file and all embedded images — can be sent over a single TCP connection, avoiding the repeated connection-setup overhead of the non-persistent case.

2.3 HTTP Message Format

There are two types of HTTP messages: request messages and response messages.

2.3.1 HTTP Request Message

A Typical HTTP Request Message

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
(blank line)
```

The first line is the request line; the lines that follow are header lines. The request line has three fields: the method field, the URL field and the HTTP-version field. The method field commonly takes values GET, POST, HEAD, PUT and DELETE.

- Host: the host on which the object resides.
- Connection: close — tells the server not to bother with a persistent connection; it should close after sending the requested object.
- User-agent: identifies the browser type making the request to the server.
- Accept-language: indicates the user's preferred language version of the object, if one exists on the server.

2.3.2 HTTP Response Message

A Typical HTTP Response Message

```
HTTP/1.1 200 OK
Connection: close
Date: Tue, 18 Aug 2015 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
Content-Length: 6821
```

```
Content-Type: text/html
(blank line)
...data data data...  <-- entity body
```

A response message has three sections: an initial status line, header lines, and then the entity body — the requested object itself. The status line has three fields: protocol version, a status code, and a corresponding status message (e.g. 200 OK).

- Connection: close — the server will close the TCP connection after this message.
- Date — the time the response was generated and sent by the server (not when the object itself was created/modified).
- Server — identifies the Web-server software, analogous to User-agent in the request.
- Last-Modified — the time the object was created or last modified; critical for caching.
- Content-Length — the number of bytes in the object being sent.
- Content-Type — officially indicates the object's MIME type (e.g. text/html), not the file extension.

Status Code	Meaning
200 OK	Request succeeded; the object is in this response.
301 Moved Permanently	Requested object has moved; new location given in a Location: header.
400 Bad Request	Generic error code — request could not be understood by the server.
404 Not Found	The requested object does not exist on the server.
505 HTTP Version Not Supported	The requested HTTP protocol version is not supported by the server.

2.4 User-Server Interaction: Cookies

DEFINITION — Cookie

A small piece of data (defined in RFC 6265) that lets a Web site keep track of a user across visits and requests, compensating for HTTP's stateless design.

Cookie technology has four components:

- A Set-Cookie header line in the HTTP response message.
- A Cookie header line in the subsequent HTTP request messages.
- A cookie file kept on the user's end system, managed by the browser.
- A back-end database maintained at the Web site.

EXAM TIP

A frequently asked short question: “Why does HTTP need cookies if it is stateless?” — answer with the definition above plus the four components.

2.5 Web Caching

DEFINITION — Web Cache / Proxy Server

A network entity that satisfies HTTP requests on behalf of an origin Web server, keeping its own disk storage of recently requested objects.

How a Web Cache Serves a Request

1. Browser -> TCP connection -> Web cache; sends HTTP request for the object to the cache.
2. Cache checks local storage:
 - HIT: cache returns the object directly to browser.
 - MISS: cache opens a TCP connection to the ORIGIN server, requests the object there.
3. Origin server sends the object to the cache.
4. Cache stores a local copy AND forwards it to the browser over the existing cache->browser connection.

Note: a cache is a SERVER to the browser and a CLIENT to the origin server, at the same time.

Typically an ISP purchases and installs Web caches — for example, a university might install a cache on its campus network and configure all campus browsers to use it. Web caching is deployed for two main reasons: first, it substantially reduces response time for a client request, particularly when the client-cache link is much faster than the client-origin-server link; second, it substantially reduces traffic on an institution's access link to the Internet, delaying the need for expensive bandwidth upgrades and improving performance for everyone.

Content Distribution Networks (CDNs) extend this idea: a CDN company installs many geographically distributed caches throughout the Internet, localising much of the traffic. There are shared CDNs (e.g. Akamai, Limelight) used by multiple content providers, and dedicated/private CDNs (e.g. Google, Netflix) built for a single provider's own content.

2.6 The Conditional GET

DEFINITION — Conditional GET

An HTTP GET request that includes an If-Modified-Since: header line, letting a cache verify whether its cached copy of an object is still up to date (RFC 7232).

Caching introduces a new problem: the cached copy of an object may be stale if the object on the origin server has since been modified. With a conditional GET, the cache sends an If-Modified-Since header carrying the date of its cached copy. If the object has not changed since that date, the origin server replies with a compact 304 Not Modified response instead of resending the entire object, saving bandwidth and time.

2.7 HTTP/2 and Framing

The primary goals of HTTP/2 are to reduce perceived latency by enabling request and response multiplexing over a single TCP connection, to provide request prioritisation and server push, and to compress HTTP header fields efficiently. Importantly, HTTP/2 does not change HTTP methods, status codes, URLs, or header fields — it changes only how data is formatted and transported between client and server.

DEFINITION — Head-of-Line (HOL) Blocking

A delay that occurs when a single lost or slow object at the front of a queue blocks all objects behind it from

being delivered, even though they are ready.

The HTTP/2 solution to HOL blocking is to break each message into small frames and interleave the request and response frames over the same TCP connection. This is performed by the framing sub-layer: when a server sends a response, the framing sub-layer breaks it into a header frame plus one or more body frames, interleaves these with the frames of other in-flight responses, and sends them all over one persistent TCP connection — the client reassembles the frames at the other end. This ability to break a message into independent frames, interleave them, and reassemble them is the single most important enhancement HTTP/2 brings.

Feature	HTTP/1.1	HTTP/2
Connections needed for many objects	Multiple, or one persistent connection used serially	One connection, fully multiplexed
Head-of-line blocking	Present	Solved via frame interleaving
Header compression	No	Yes
Server push	No	Yes
Methods, status codes, URLs, headers	Baseline	Unchanged from HTTP/1.1

3. Electronic Mail in the Internet

Like ordinary postal mail, e-mail is an asynchronous communication medium — people send and read messages when it is convenient for them, without coordinating schedules with the other party. Unlike postal mail, electronic mail is fast, easy to distribute and inexpensive, and modern e-mail supports attachments, hyperlinks, HTML-formatted text and embedded photos.

3.1 The Three Major Components

Internet e-mail has three major components: user agents, mail servers, and the Simple Mail Transfer Protocol (SMTP).

- **User Agents:** allow users to read, reply to, forward, save and compose messages. Examples: Microsoft Outlook, Apple Mail, Web-based Gmail, the Gmail app.
- **Mail Servers:** form the core of the e-mail infrastructure. Each recipient has a mailbox managed on their mail server; the sender's messages sit in an outgoing message queue on the sender's mail server until they can be delivered.
- **SMTP:** the application-layer protocol that transfers messages from the sender's mail server to the recipient's mail server.

Path of a Message from Alice to Bob

Alice's user agent (compose)	->	Alice's mail server (outgoing queue)	-->	Bob's mail server (SMTP push, port 25)	->	Bob's user agent (reads from his mailbox)
------------------------------------	----	---	-----	---	----	--

3.2 SMTP

DEFINITION — SMTP (Simple Mail Transfer Protocol)

The principal application-layer protocol for Internet electronic mail (RFC 5321); it transfers messages from a sender's mail server to a recipient's mail server using TCP's reliable data transfer service.

SMTP is much older than HTTP. It has both a client side (running on the sending mail server) and a server side (running on the receiving mail server); both sides run on every mail server — a mail server acts as an SMTP client when sending mail to other servers, and as an SMTP server when receiving mail. Importantly, SMTP does not normally use intermediate mail servers, even when the two communicating mail servers are on opposite sides of the world.

The client SMTP (on the sending mail server) has TCP establish a connection to port 25 on the server SMTP (on the receiving mail server). If the server is down, the client tries again later. Once connected, client and server perform application-layer handshaking, exchanging some initial information. SMTP uses persistent connections — if the sending mail server has several messages queued for the same receiving mail server, it can send them all over the same TCP connection.

KEY POINT

SMTP has notable similarities to protocols used for face-to-face human interaction — it performs a conversational handshake before any 'message' (mail data) is actually transferred.

3.3 Mail Message Formats

Every e-mail header must include a From: header line and a To: header line; it may also include a Subject: line and other optional header lines. These header lines, part of the mail message itself, are distinct from the SMTP handshaking commands, even though some share common words such as “from” and “to”.

COMMON CONFUSION

Do not confuse SMTP commands (part of the handshaking protocol between mail servers) with mail message header lines (From:, To:, Subject: — part of the message content itself). Examiners often test this distinction directly.

3.4 Mail Access Protocols

SMTP is a push protocol — it is used only to move mail from a sender's mail server to a recipient's mail server, and it cannot be used by the recipient's user agent to pull messages out of the mailbox. A separate mail access protocol is needed for the final step: transferring mail from the recipient's mail server to the recipient's own device.

Protocol	Nature	Key Characteristic
POP3 (Post Office Protocol, v3)	Mail access protocol	Simple; typically downloads mail to the local device and (by default) deletes it from the server — limited support for maintaining folders across sessions.
IMAP (Internet Mail Access Protocol)	Mail access protocol	More complex and feature-rich; keeps all messages in one place (the server), letting a user organise messages into server-side folders and access them consistently from multiple devices.
HTTP-based Webmail	Mail access via a browser	The user agent is a Web browser and the user communicates with a remote mailbox via HTTP (e.g. Gmail's Web interface).

EXAM TIP

A typical question: “Why is SMTP alone not enough for e-mail?” — answer: SMTP only pushes mail between mail servers; a mail access protocol (POP3/IMAP/HTTP) is needed to pull the mail from the recipient's server to the recipient's own device.

4. DNS — The Internet's Directory Service

One identifier for a host is its hostname — e.g. `www.facebook.com`, `gaia.cs.umass.edu`. Hostnames are mnemonic and appreciated by humans, but they provide little information about the host's location within the network. Meanwhile, an IP address consists of four bytes with a rigid hierarchical structure, e.g. `121.7.106.83`, where each period separates a byte expressed in decimal (0–255). Routers use IP addresses efficiently, but humans cannot easily remember them — this is precisely the gap the Domain Name System (DNS) fills.

4.1 Services Provided by DNS

DEFINITION — DNS (Domain Name System)

(1) A distributed database implemented in a hierarchy of DNS servers, and (2) an application-layer protocol that allows hosts to query that distributed database in order to translate hostnames to IP addresses.

DNS servers are frequently UNIX machines running the Berkeley Internet Name Domain (BIND) software. The DNS protocol runs over UDP and uses port 53.

Besides translating hostnames to IP addresses, DNS provides other important services:

- **Host aliasing:** a host with a complicated hostname (the canonical hostname) can have one or more shorter, more mnemonic alias hostnames. DNS can return either the canonical hostname or the IP address for a supplied alias.
- **Mail server aliasing:** lets a company's mail server and Web server share an identical-looking (aliased) hostname, e.g. both being reachable as `enterprise.com`, using an MX record.
- **Load distribution:** DNS performs load distribution among replicated servers — a busy site such as `cnn.com` is replicated over multiple servers, each with a different IP address; DNS can return a rotating/different address per query.

4.2 Overview of How DNS Works

When an application (e.g. a Web browser) needs to translate a hostname to an IP address, it invokes the client side of DNS, specifying the hostname to be translated (on many UNIX systems, via a call such as `gethostbyname()`). DNS on the user's host sends a query message into the network; all DNS query and reply messages travel inside UDP datagrams to port 53. After a delay ranging from milliseconds to seconds, DNS on the user's host receives a reply message with the desired mapping, which is passed back to the invoking application. From the application's perspective, DNS is simply a black box offering a straightforward translation service.

COMMON CONFUSION

A single centralized DNS server sounds simple but is impractical for today's Internet because of: a single point of failure (if it crashes, the entire lookup service dies), unmanageable traffic volume, huge propagation delay for distant clients, and constant maintenance of a gigantic record set.

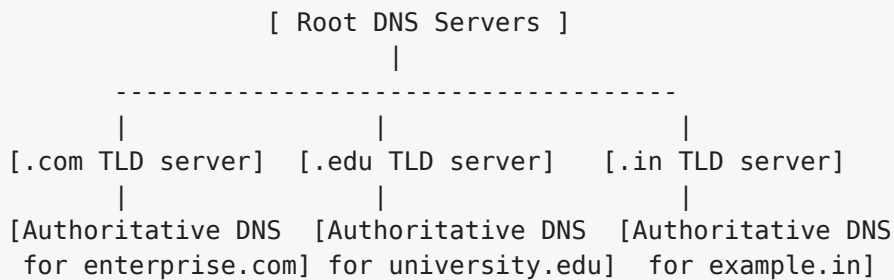
4.3 A Distributed, Hierarchical Database

To deal with scale, DNS uses a large number of servers, organised hierarchically and distributed worldwide. No single DNS server holds the mappings for every host on the Internet; instead, mappings are distributed across DNS servers. To a first approximation, there are three classes of DNS servers, organised in a hierarchy:

- Root DNS servers: at the top of the hierarchy; they know how to reach the TLD servers.
- Top-Level Domain (TLD) DNS servers: responsible for domains such as .com, .org, .net, .edu, and country codes such as .in, .uk, .fr.
- Authoritative DNS servers: hold the actual hostname-to-IP mappings for the hosts within an organisation — every organisation with publicly accessible hosts must provide these records, either self-hosted or hosted by a service provider.



DNS Server Hierarchy (simplified)



4.4 DNS Caching



DEFINITION — DNS Caching

The practice by which a DNS server, having received a DNS reply (e.g. a hostname-to-IP mapping), stores that mapping in its local memory for future use.

DNS extensively exploits caching to improve delay performance and to reduce the volume of DNS messages travelling around the Internet. Because host-to-IP mappings are not permanent, cached entries are timed out (discarded) after a certain length of time, defined by the record's TTL (time-to-live).

4.5 DNS Records and Messages

The DNS servers that together implement the DNS distributed database store resource records (RRs), including the RRs that provide hostname-to-IP-address mappings. Every DNS reply message carries one or more resource records.



DEFINITION — Resource Record (RR)

A four-tuple (Name, Value, Type, TTL) stored by DNS servers, whose meaning depends on the Type field.

Type	Meaning of Name	Meaning of Value	Example
A	A hostname	The IP address for that hostname	(relay1.bar.foo.com, 145.37.93.126, A)
NS	A domain (e.g. foo.com)	Hostname of an authoritative DNS server that knows how to obtain IP addresses for hosts in that domain	(foo.com, dns.foo.com, NS)
CNAME	An alias hostname	The canonical (true) hostname for that alias	(foo.com, relay1.bar.foo.com, CNAME)
MX	An alias hostname	The canonical name of a mail server	(foo.com, mail.bar.foo.com, MX)

 KEY POINT

MX records let a company use the SAME aliased name for its mail server and, say, its Web server (both can be “enter prise.com”). To get the mail server's canonical name a client queries for an MX record; to get another server's canonical name it queries for a CNAME record.

4.5.1 DNS Message Format

The first 12 bytes of a DNS message form the header section. It includes a 16-bit identifier (copied from query to reply so the client can match replies to queries it sent) and several flag bits, including a 1-bit query/reply flag (0 = query, 1 = reply).

- Question section: contains a name field (the name being queried) and a type field (e.g. Type A for a host address, or Type MX for a mail server).
- Answer section (in a reply): contains the resource record(s) for the name originally queried — a reply can carry multiple RRs, since one hostname can map to multiple IP addresses (e.g. replicated Web servers).
- Authority section: contains records of other authoritative servers.
- Additional section: contains other helpful records — e.g. the answer to an MX query returns the mail server's canonical hostname, and the additional section supplies a Type A record giving that hostname's IP address.

 EXAM TIP

A very common long-answer question: “Explain the DNS resource record format and describe the four types A, NS, CNAME, MX with an example each.” Reproduce the table above exactly.

5. Peer-to-Peer (P2P) File Distribution

Consider distributing a single large file from one server to N other peers. In client-server file distribution, the server must send a separate copy of the file to each of the N peers, placing an enormous burden on the server and consuming a large amount of server bandwidth — distribution time grows (at best) linearly with N . In P2P file distribution, every peer can redistribute any portion of the file it has already received to other peers, assisting the server rather than only drawing from it.

5.1 Scalability of P2P Architectures

KEY POINT

The defining strength of P2P file distribution is self-scalability: each new peer that arrives both consumes capacity (by downloading) and adds capacity (by uploading to others). This is the opposite of client-server distribution, where each new client purely adds load to a fixed server.

5.2 BitTorrent

DEFINITION — BitTorrent

A popular P2P protocol for file distribution in which the collection of all peers participating in the distribution of one particular file is called a torrent.

Peers in a torrent download equal-size chunks of the file from one another, with a typical chunk size of 256 KBytes. When a peer first joins a torrent it has no chunks, but over time it accumulates more; while it downloads chunks it simultaneously uploads chunks to other peers. Once a peer has acquired the entire file, it may selfishly leave the torrent, or altruistically remain and continue uploading to others. Any peer may also leave with only a subset of chunks and rejoin later.

5.2.1 The Tracker

DEFINITION — Tracker

An infrastructure node belonging to a torrent; when a peer joins the torrent it registers itself with the tracker and periodically informs it that it is still active, so the tracker keeps track of participating peers.

A given torrent may have anywhere from fewer than ten to more than a thousand peers participating at any instant.

5.2.2 How a Peer Obtains Chunks

Chunk Exchange (Alice's Perspective)

1. Alice periodically asks each of her L neighbouring peers (over existing TCP connections) which chunks they have -> she gets L chunk-lists.
2. Using this knowledge, Alice requests (over the same TCP connections) the chunks she does not yet have.
3. While downloading, Alice ALSO uploads chunks she already has to some of her neighbours.

In practice, a peer typically requests the rarest chunks among its neighbours first ("rarest first"), which helps keep every chunk well represented across the swarm and reduces the risk that a chunk disappears if the only peer holding it leaves.

To decide who to upload to, a peer uses a tit-for-tat-style incentive: it prioritises sending chunks to the neighbours that are currently supplying it data at the highest rate, periodically re-evaluating this small set of favoured neighbours (“unchoking” them) and occasionally unchoking a random peer as well, to discover potentially better partners. This rewards peers who reciprocally upload and discourages free-riding (downloading without ever uploading).

**EXAM TIP**

Be ready to explain, in your own words, why P2P scales better than client-server for large-file distribution — examiners often ask for a comparison of total distribution time as the number of downloaders N grows.

6. Video Streaming and Content Distribution Networks

Video traffic is the dominant consumer of Internet bandwidth today. Video streaming has two defining properties that shape how it must be engineered: it produces a high, roughly continuous, data rate, and it is loss-tolerant but delay-sensitive within limits — a viewer wants a smooth, timely playback more than perfect bit-for-bit accuracy.

6.1 A Naive (Single-Data-Centre) Approach and Its Problems

The most straightforward approach to providing streaming video service is to build one single massive data centre, store all videos there, and stream them directly to clients worldwide. This runs into three major problems:

- **Distance:** if a client is far from the data centre, server-to-client packets cross many communication links and possibly many ISPs, some on different continents — raising delay and the chance of a slow link along the path.
- **Bottleneck throughput:** if any one link on the path provides throughput below the video's consumption rate, end-to-end throughput drops below that rate too, causing the viewer's playback to freeze.
- **A single point of load:** the one data centre must simultaneously serve every client on Earth, multiplying the above problems under peak load.

To meet the challenge of distributing massive amounts of video data worldwide, almost all major video-streaming companies use Content Distribution Networks (CDNs) instead.

6.2 HTTP Streaming and DASH

DEFINITION — DASH (Dynamic Adaptive Streaming over HTTP)

A video-streaming technique in which a video is encoded into several different quality/bit-rate versions, and the client dynamically requests chunks of whichever version best matches its currently available bandwidth.

With HTTP streaming, the video is simply stored on an HTTP server as an ordinary file with a specific URL; a client requests it via an HTTP GET, and the server sends it back inside an HTTP response message. DASH builds on this by storing each video as several chunked, different-bit-rate versions on the server; the client measures its own available bandwidth in real time and requests the next chunk from whichever version fits best — seamlessly switching quality up or down (adapting) as network conditions change, which is why it is called adaptive streaming.

6.3 Content Distribution Networks (CDNs)

DEFINITION — Content Distribution Network (CDN)

A system that manages servers in multiple geographically distributed locations, stores copies of videos and other Web content in those servers, and attempts to direct each user request to the CDN location expected to give the best user experience.

A CDN may be private, owned by the content provider itself (e.g. Google's CDN, which distributes YouTube videos), or it may be a third-party CDN that distributes content on behalf of multiple, unrelated content providers (e.g. Akamai, Limelight, Level-3).

6.3.1 CDN Operation

When a client requests a piece of content, the CDN must intercept that request so that it can (1) determine a suitable CDN server cluster for that client at that moment, and (2) redirect the client's request to a server within that cluster. Most CDNs use DNS to intercept and redirect requests: because the client must first perform a DNS lookup to obtain the IP address of the requested content, the CDN's own DNS infrastructure can return the IP address of a nearby/suitable cluster instead of the origin server's own address.

6.3.2 Cluster Selection Strategies

DEFINITION — Cluster Selection Strategy

A mechanism used by a CDN to dynamically direct a client to one particular server cluster (or data centre) among its many distributed locations.

A CDN typically learns the IP address of the client's local DNS (LDNS) server via the client's DNS lookup, and uses that as an approximate proxy for the client's own network location when choosing a nearby cluster. Common strategies include geographically closest cluster, and measured real-time network conditions/load-based selection, so that the chosen cluster is likely to give low delay and high throughput to that specific client.

6.4 Case Studies: Netflix and YouTube

Aspect	Netflix	YouTube
CDN ownership	Uses its own private CDN infrastructure for video delivery	Uses Google's private CDN infrastructure
Control/upload pipeline	Licensed movies/shows ingested and pushed out to CDN servers	User-generated content uploaded continuously by the public
Content catalogue size	Large but curated / relatively stable	Enormous and constantly growing
Key engineering challenge	Pre-positioning a known, finite catalogue close to users	Handling a huge, ever-changing, long-tail catalogue at scale

EXAM TIP

If asked to 'explain how a CDN improves video streaming performance', structure the answer as: (1) problem with a single data centre, (2) definition of a CDN, (3) how DNS is used to redirect the client, (4) benefit — lower delay, higher throughput, less core-network congestion.

7. Socket Programming

Socket programming is the practical, hands-on side of everything covered so far: it is how a developer actually builds the client and server processes that use TCP or UDP to exchange application data. A socket, recall, is the door through which application data passes between a process and the underlying transport layer. Because the two transport protocols behave so differently, the sequence of socket calls a program must make is different for UDP and for TCP.

7.1 Socket Programming with UDP

KEY POINT

UDP is connectionless — there is no handshaking before data is exchanged, and no ‘pipe’ is ever set up between client and server. The sender must attach the destination's IP address and port number to every single packet it sends, because each packet is independent.

UDP Socket Programming — Typical Sequence

SERVER	CLIENT
-----	-----
create socket (UDP)	create socket (UDP)
bind socket to a port	(no bind needed)
wait, recvfrom() ...	sendto(server_ip, port, msg)
<----- datagram -----	
process message	
sendto(client_ip, client_port, reply)	
----- datagram ----->	
	recvfrom() -> reply
(loop / close)	close socket

Because UDP is connectionless, the server does not need to ‘accept’ a connection — it simply waits on a bound port for incoming datagrams. Every datagram that arrives at `recvfrom()` also reveals the sender's IP address and port, which the server needs in order to know where to send its reply. Because delivery is unreliable, an application built on UDP must implement its own acknowledgement/retransmission logic if it needs reliability, and it must also handle messages that may arrive out of order or not at all.

7.2 Socket Programming with TCP

KEY POINT

TCP is connection-oriented — client and server must first complete a handshake to establish a connection before any application data flows. Once established, this connection behaves like a reliable, ordered, full-duplex byte stream, and the socket calls reflect that extra structure.

TCP Socket Programming — Typical Sequence

SERVER	CLIENT
-----	-----
create socket (TCP)	
bind socket to a port	
listen() for connections	
accept() -> blocks until a client connects	create socket (TCP)
	connect(server_ip, port)

```

<---- 3-way TCP handshake completes ---->
accept() returns a NEW,
connection-specific socket
recv() / send() <-----> send() / recv()
close(connection socket)    close(socket)

```

On the server side, `listen()` puts the original 'welcome' socket into a state where it can accept incoming connection requests, and `accept()` blocks until a client actually connects — at which point it returns a brand-new socket dedicated to that one client, leaving the welcome socket free to accept further connections from other clients. On the client side, `connect()` triggers the TCP three-way handshake with the server. Once the handshake completes, both sides simply `send()` and `recv()` bytes over the now-reliable connection; TCP itself guarantees in-order, error-free delivery, so the application does not need to implement its own acknowledgement scheme.

Aspect	UDP Sockets	TCP Sockets
Connection setup	None — send directly with <code>sendto()</code>	<code>listen()</code> / <code>accept()</code> (server), <code>connect()</code> (client)
Destination per packet	Must be given with every <code>sendto()</code> call	Given once at <code>connect()</code> time; not needed again
Reliability	Application must add its own, if needed	Guaranteed in-order, error-free delivery by TCP
Server handling multiple clients	One socket serves everyone, using sender info per packet	A new socket is returned by <code>accept()</code> for each client
Typical primitives	<code>socket</code> , <code>bind</code> , <code>sendto</code> , <code>recvfrom</code> , <code>close</code>	<code>socket</code> , <code>bind</code> , <code>listen</code> , <code>accept</code> , <code>connect</code> , <code>send</code> , <code>recv</code> , <code>close</code>



COMMON CONFUSION

A very common exam mix-up: students say the server 'listens on the same socket it uses to talk to the client' for TCP. In fact, TCP's `accept()` returns a brand-new, connection-specific socket for each client, while the original welcome socket stays free to accept more connections.



EXAM TIP

Be ready to draw the two sequence diagrams above from memory and to explain, in one line each, what `bind()`, `listen()`, `accept()` and `connect()` do.

8. Quick Revision — Master Summary Tables

This chapter is designed for the night before the exam: every important comparison from the earlier chapters is collected here in one place, with nothing else to read around it.

8.1 Application → Protocol → Port → Transport

Application	Protocol	Well-Known Port	Transport
Web	HTTP	80	TCP
Secure Web	HTTPS	443	TCP
E-mail (sending)	SMTP	25	TCP
Remote terminal	Telnet	23	TCP
File transfer	FTP	21 (control)	TCP
Directory service	DNS	53	UDP (mostly)
Internet telephony	SIP / RTP	varies	UDP or TCP

8.2 Client-Server vs P2P (one-line memory hooks)

- Client-Server: always-on server, fixed IP, clients never talk to each other directly.
- P2P: self-scalable, no dedicated server needed, security/reliability is harder to guarantee.

8.3 TCP vs UDP (one-line memory hooks)

- TCP = connection + reliable + congestion-controlled + slower.
- UDP = connectionless + unreliable + no congestion control + faster, lighter.

8.4 HTTP Essentials

Item	Value / Fact
Transport used by HTTP	TCP (always)
Default port	80 (HTTP), 443 (HTTPS)
HTTP/1.0 default connection type	Non-persistent
HTTP/1.1 default connection type	Persistent
Header that requests a fresh copy only if changed	If-Modified-Since (Conditional GET)
Mechanism that lets a stateless HTTP track users	Cookies (RFC 6265)
HTTP/2's main fix	Frame interleaving — removes head-of-line blocking

8.5 DNS Essentials

Item	Value / Fact
Transport used by DNS	UDP, port 53
Three server classes	Root → TLD → Authoritative
A record	hostname → IP address
NS record	domain → hostname of an authoritative DNS server
CNAME record	alias hostname → canonical hostname
MX record	alias hostname → canonical name of a mail server
Why caching is used	Reduces delay + reduces DNS message volume

8.6 E-mail Essentials

Item	Value / Fact
Protocol that pushes mail server-to-server	SMTP, port 25, over TCP
Protocols that pull mail to the user's device	POP3, IMAP, or HTTP-based Webmail
Mandatory message header lines	From: and To:
SMTP connection type	Persistent (can send several queued messages in one session)

8.7 P2P / BitTorrent Essentials

Item	Value / Fact
Group of peers sharing one file	A torrent
Infrastructure node tracking who's in the torrent	The tracker
Typical chunk size	256 KBytes
Chunk-selection strategy	Rarest-first
Upload-priority strategy	Tit-for-tat (reward highest-rate uploaders)

8.8 Video / CDN Essentials

Item	Value / Fact
Adaptive streaming over HTTP	DASH
What a CDN stores	Copies of content at many geographically distributed servers
Mechanism CDNs use to redirect clients	DNS-based interception/redirection
Private CDN example	Google (YouTube), Netflix
Third-party CDN example	Akamai, Limelight

**EXAM TIP**

If you can reproduce every table in this chapter from memory, you can answer almost every 2-mark and 5-mark question this unit typically carries. Use Chapter 9 to practise turning these facts into full exam answers.

9. Bloom's Taxonomy Question Bank

Attempt every question below without looking back at the earlier chapters first, then check your answer against the model answer given. The questions climb, level by level, from simple recall to original design — exactly the range a good exam paper draws from.

9.1 Remember — Recall Facts & Definitions

LEVEL: REMEMBER — *Can you state it correctly, from memory?*

Q1. Define client-server architecture.

Ans: An architecture with an always-on host (the server) that services requests from many other hosts (clients); clients do not communicate directly with each other.

Q2. Define P2P architecture.

Ans: An architecture with minimal or no reliance on dedicated servers, where pairs of intermittently connected hosts called peers communicate directly with one another.

Q3. What is a socket?

Ans: The software interface between the application layer and the transport layer within a host — the API through which a process sends and receives network messages.

Q4. Define reliable data transfer.

Ans: A transport-layer guarantee that data sent by one end of an application is delivered correctly and completely to the other end.

Q5. What is a non-persistent HTTP connection?

Ans: A TCP connection that is closed by the server immediately after it sends one requested object, so each object needs its own connection.

Q6. What is an HTTP cookie?

Ans: A small piece of data (RFC 6265) that lets a Web site track a user across requests, using a header in the response, a header in later requests, a client-side cookie file, and a server-side database.

Q7. Define DNS.

Ans: A distributed database implemented in a hierarchy of DNS servers, plus an application-layer protocol that lets hosts query that database, mainly to translate hostnames into IP addresses.

Q8. What is a torrent (in BitTorrent terminology)?

Ans: The collection of all peers participating in the distribution of one particular file.

Q9. What is a Content Distribution Network (CDN)?

Ans: A system of servers deployed in multiple geographic locations that stores copies of content and directs each user's request to the location expected to give the best experience.

Q10. Define the conditional GET.

Ans: An HTTP GET request carrying an If-Modified-Since header, used by a cache to check whether its stored copy of an object is still up to date.

9.2 Understand — Explain in Your Own Words

LEVEL: UNDERSTAND — *Can you explain WHY, not just WHAT?*

Q1. Explain why P2P architectures are described as self-scalable.

Ans: Because each peer both consumes capacity (by downloading) and supplies capacity (by uploading to others). As more peers join, total system capacity grows alongside total demand, unlike a client-server system where new clients only add load to a fixed server.

Q2. Explain the role of a socket as the interface between a process and the network.

Ans: A process has full control over the application side of the socket (what to write into it, when) but almost no control over the transport side; the socket is simply where the application layer hands data off to the transport layer (and vice versa) for delivery across the network.

Q3. Explain the four dimensions along which transport services can be classified.

Ans: Reliable data transfer (guaranteed, complete, correct delivery), throughput (rate of successful data delivery), timing (bounded end-to-end delay), and security (confidentiality, integrity, authentication) — different applications need different combinations of these.

Q4. Explain how TCP's and UDP's handling of congestion differs.

Ans: TCP includes a congestion-control mechanism that throttles the sender when the network is congested, protecting the network's general welfare. UDP has no congestion control at all — it sends at whatever rate the application requests, regardless of network state.

Q5. Explain how a Web page actually gets delivered once a browser sends an HTTP GET.

Ans: The browser first establishes a TCP connection to the server's port 80 (or reuses a persistent one), sends the HTTP request into its socket, the server retrieves the object and sends back an HTTP response through its own socket, and TCP guarantees the bytes arrive at the browser in order and intact.

Q6. Explain why cookies had to be introduced for HTTP.

Ans: HTTP is stateless by design — the server keeps no memory of earlier requests from the same client. Sites that need to recognise returning users (shopping carts, logins, personalisation) therefore need an external mechanism, which cookies provide.

Q7. Explain how DNS caching helps both delay and traffic volume.

Ans: Once a DNS server has resolved a hostname, it stores the mapping locally. Later queries for the same hostname can be answered instantly from the cache instead of being sent all the way up the DNS hierarchy again, which reduces both the response delay for that query and the total number of DNS messages travelling through the Internet.

Q8. Explain what specific problem persistent HTTP connections solve.

Ans: Non-persistent connections force a brand-new TCP connection (with its own handshake and buffer allocation) for every single object on a page. Persistent connections reuse one TCP connection for the base HTML file and all its embedded objects, avoiding this repeated setup cost.

Q9. Explain why SMTP by itself cannot deliver mail to a user's phone or laptop.

Ans: SMTP is a push protocol designed only to move mail from a sending mail server to a receiving mail server; it cannot be used by a user's own device to pull messages out of a mailbox. A separate mail access protocol (POP3, IMAP, or HTTP webmail) is required for that final hop.

Q10. Explain, in outline, how HTTP/2 solves head-of-line blocking.

Ans: HTTP/2's framing sub-layer breaks every message (header and body) into small independent frames, interleaves frames from multiple in-flight requests/responses over the single TCP connection, and reassembles them at the receiving end — so one slow object no longer blocks all the others behind it.

9.3 Apply — Use the Concept in a New Situation

LEVEL: APPLY — *Can you use it to solve something you haven't seen before?*

Q1. A video-conferencing app needs low delay and can tolerate a little packet loss. Which transport protocol would you choose, and why?

Ans: UDP — it has no handshaking delay and no congestion-control throttling, giving the lowest possible delay, and the application is loss-tolerant so UDP's lack of reliability is an acceptable trade-off. (In practice, apps often fall back to TCP only if UDP is blocked.)

Q2. A page has 1 HTML file and 5 images. Roughly how many RTTs does it take to load fully under non-persistent HTTP/1.0 versus persistent HTTP/1.1 (ignore transmission time, assume no parallel connections)?

Ans: Non-persistent: about 2 RTTs per object (handshake + request/response) $\times$ 6 objects $\approx$ 12 RTTs. Persistent: 2 RTTs for the first handshake + request/response, then about 1 RTT per remaining object over the same open connection $\approx 2 + 5 = 7$ RTTs — noticeably fewer.

Q3. A company wants its mail server and Web server to be reachable under the same friendly alias, 'enterprise.com'. Which DNS records would you configure?

Ans: An MX record mapping enterprise.com to the mail server's canonical hostname (used when resolving mail delivery), and a CNAME record mapping enterprise.com to the Web server's canonical hostname (used when resolving Web browsing) — the same alias resolves differently depending on which record type is queried.

Q4. A developer wants a file-sharing service that should scale automatically as more users join, without them buying more servers. Which architecture fits, and why?

Ans: P2P — because it is self-scalable: each new peer adds both demand and supply (upload) capacity, so the system's total capacity grows with its user base instead of being capped by fixed server hardware.

Q5. List, in order, the socket-API calls a TCP server must make to accept and serve one client.

Ans: socket() $\rightarrow$ bind() $\rightarrow$ listen() $\rightarrow$ accept() (returns a new, connection-specific socket) $\rightarrow$ recv()/send() to exchange data $\rightarrow$ close() the connection socket.

Q6. A browser sends a conditional GET with If-Modified-Since, and the object hasn't changed since that date. What does the server return, and what does this save?

Ans: The server returns a 304 Not Modified response with no entity body, instead of resending the whole object — saving bandwidth and reducing response time, while the browser simply reuses its already-cached copy.

Q7. In a BitTorrent swarm, a peer's neighbours between them have every chunk except chunk 7, which only one neighbour has. Which chunk should the peer request first, and why?

Ans: Chunk 7 — following the rarest-first strategy, requesting the rarest chunk first reduces the risk that it becomes completely unavailable if its only holder leaves the torrent, and helps that chunk spread more evenly across the swarm.

Q8. A user in India streams a YouTube video originally hosted on a US server. Using CDN concepts, explain how delay is reduced for this user.

Ans: Google's private CDN keeps cached copies of popular videos on servers geographically close to India. When the user's player resolves the video URL, DNS-based redirection points it to a nearby CDN cluster instead of the distant US origin server, cutting the number of long-haul links the video data has to cross and reducing both delay and the risk of a bottleneck link.

9.4 Analyze — Compare and Break Down

LEVEL: ANALYZE — *Can you break it into parts and see how they relate or differ?*

Q1. Compare TCP and UDP across connection setup, reliability, congestion control, and typical use.

Ans: TCP: handshake required, reliable in-order delivery, has congestion control, used by HTTP/SMTP/FTP. UDP: no handshake, unreliable/unordered delivery, no congestion control, used by DNS and real-time media where speed matters more than guarantees.

Q2. Differentiate client-server and P2P architecture in terms of scalability and security.

Ans: Scalability: client-server needs the operator to manually add server capacity as demand grows; P2P scales itself because peers add capacity as they join. Security: client-server is easier to secure and monitor centrally; P2P is harder to secure because it is decentralised and peers are not controlled by a single operator.

Q3. Differentiate POP3 and IMAP.

Ans: POP3 is simple and typically downloads mail to one device, by default removing it from the server, giving limited multi-device consistency. IMAP is more feature-rich, keeps mail organised in folders on the server itself, and lets a user access the same consistent view of their mailbox from multiple devices.

Q4. Compare non-persistent and persistent HTTP connections in terms of RTT cost and TCP overhead.

Ans: Non-persistent connections cost roughly 2 RTTs and a fresh TCP handshake/buffer allocation per object; persistent connections pay that cost once and then only about 1 RTT per additional object over the same open connection, substantially reducing overhead for pages with many embedded objects.

Q5. Differentiate the DNS record types A, NS, CNAME and MX.

Ans: A maps a hostname to an IP address. NS maps a domain to the hostname of an authoritative server for that domain (used to route the query further). CNAME maps an alias hostname to its canonical hostname. MX maps an alias hostname to the canonical name of a mail server for that domain.

Q6. Compare a Web cache (proxy server) with a CDN in terms of scope and deployment.

Ans: A Web cache is typically a single, local deployment (e.g. one machine at a university or ISP) serving a specific community of client browsers. A CDN is a globally distributed system of many caches/clusters, owned by (or on behalf of) a content provider, designed to serve any client worldwide from a nearby location.

Q7. Analyze why HTTP is called 'stateless' and what implication this has for application design.

Ans: HTTP is stateless because the server, by protocol design, keeps no record of a client's earlier requests — each request is handled independently. This means any application feature that needs memory of the user across requests (login sessions, shopping carts) must be engineered on top of HTTP using an external mechanism such as cookies or server-side session IDs.

Q8. Compare HTTP/1.1 and HTTP/2 in how they handle multiple objects on one connection.

Ans: HTTP/1.1's persistent connection sends objects largely one after another (or pipelines them, still subject to head-of-line blocking), so a slow object can delay everything behind it. HTTP/2 breaks every message into frames and interleaves frames from multiple requests/responses over the same connection, so objects are delivered concurrently and one slow object no longer blocks the others.

9.5 Evaluate — Judge and Justify

LEVEL: EVALUATE — *Can you take a position and defend it with evidence?*

Q1. Is P2P always a better choice than client-server for file distribution? Justify with at least one point on each side.

Ans: Not always. For: P2P scales distribution time far better as the number of downloaders grows, since peers redistribute chunks to each other instead of all pulling from one server. Against: client-server gives far tighter control over security, content integrity and availability guarantees, which matters for, say, a bank's file downloads — the right choice depends on whether raw scalability or centralised control/trust matters more for that application.

Q2. A developer claims UDP should never be used because it is 'unreliable'. Evaluate this claim.

Ans: The claim is too strong. 'Unreliable' only means UDP itself gives no delivery guarantee — for loss-tolerant, delay-sensitive applications such as live video calls or DNS lookups, UDP's low overhead and absence of congestion-throttling are actual advantages, and any reliability the application does need can be added at the application layer. UDP is the wrong choice only for applications (file transfer, e-mail) that cannot tolerate any data loss.

Q3. Justify why DNS is built as a distributed, hierarchical database rather than one centralized server.

Ans: A single centralized DNS server would be a single point of failure for the entire Internet, would have to absorb every DNS query worldwide (an unmanageable traffic volume), would be physically distant from most clients (adding delay), and would need constant updating for every host on Earth. The distributed hierarchy (root → TLD → authoritative) spreads load, keeps records close to the organisations responsible for them, and removes any single failure point.

Q4. Critically evaluate whether cookies pose a privacy risk, given how they work.

Ans: Cookies can be a genuine privacy concern: because a cookie header is echoed back on every subsequent request to the same site (and, for third-party cookies, potentially to other sites embedding that content), a site — or an advertising network spanning many sites — can build a detailed profile of a user's browsing behaviour over time, often without the user being fully aware of it. This is why modern browsers restrict third-party cookies and regulations require consent banners.

Q5. Is it justified for real-time video-conferencing apps to fall back to TCP if UDP fails, even though TCP is not ideal for real-time media?

Ans: Yes, as a pragmatic trade-off: some networks/firewalls block UDP outright, so without a TCP fallback the call would not connect at all. A slightly degraded but working TCP-based call (with more delay from retransmissions and no ability to skip lost data) is preferable to no call at all — UDP remains the preferred primary path precisely because it avoids this problem when it is available.

Q6. Evaluate the trade-off a CDN operator faces between many small server clusters versus few large ones.

Ans: Many small, widely distributed clusters minimise client-to-server distance (lower delay, better user experience) but raise operating cost and the complexity of keeping every cluster's cached content up to date. Few large clusters are cheaper to run and easier to keep synchronised, but leave some clients far away, increasing delay and the risk of a bottleneck link. The right balance depends on how delay-sensitive the content is and the provider's budget.

9.6 Create — Design Something New

LEVEL: CREATE — *Can you design or propose an original solution?*

Q1. Design a simple application-layer protocol for a basic two-user chat application.

Ans: Define two message types — CONNECT (username) and MSG (text) — each with a fixed 1-byte type field followed by a length-prefixed payload. Syntax: [type][length][payload]. Semantics: CONNECT registers a username with the server; MSG carries chat text to be relayed to the other user. Rule: a client must send exactly one CONNECT before any MSG; the server relays every MSG it receives to all other connected clients and sends an ERROR message type if a MSG arrives before CONNECT.

Q2. Propose a caching strategy for a university's Web proxy that balances freshness and bandwidth savings.

Ans: Cache every static object (images, CSS, JS) with a long local TTL, but always issue a conditional GET (If-Modified-Since) before serving anything older than, say, 24 hours, so stale-but-unlikely-to-have-changed objects are served instantly while genuinely updated objects are re-fetched. Exempt clearly dynamic content (paths containing /api/ or marked no-cache in their headers) from caching altogether.

Q3. Design the socket-level flow for a simple UDP-based echo service (server sends back whatever the client sends).

Ans: Server: `socket()` → `bind(port)` → `loop { recvfrom() to get (data, client_addr) → sendto(client_addr, data) }`.
 Client: `socket()` → `sendto(server_addr, message)` → `recvfrom()` to receive the echoed reply → `close()`. No `listen()/accept()/connect()` are needed anywhere, because UDP is connectionless.

Q4. Propose a DNS record configuration for a startup 'mystartup.io' needing a Web server, an aliased mail server, and 3 load-balanced Web servers.

Ans: Create three A records mapping `www.mystartup.io` to the three replicated Web servers' distinct IP addresses (the DNS server can rotate which one it returns, achieving simple load distribution). Create one MX record mapping `mystartup.io` to `mail.mystartup.io` (the mail server's canonical name), and a CNAME record if a shorter alias for the mail service is also wanted.

Q5. Design a chunk-request and peer-selection policy for a new P2P file-sharing app that discourages free-riding.

Ans: Chunk selection: request the rarest chunk first among a peer's known neighbours, to keep every chunk well distributed across the swarm. Peer selection for uploading: maintain a small 'favoured' set of neighbours re-evaluated periodically, prioritising those currently uploading to you at the highest rate (tit-for-tat), and occasionally unchoke a random peer to discover better partners — a peer that never uploads earns a low priority from everyone else and gets poor download performance in return.



EXAM TIP

If a question in the real exam does not obviously match one of the six levels above, ask yourself: 'Am I being asked to state it, explain it, use it, compare it, judge it, or design it?' — that single question tells you how much detail and what kind of answer is expected.

10. Important Questions — Exam Checklist

A final self-test list, grouped the way MCA question papers are usually structured: short (2-mark), medium (5-mark) and long (10-mark) questions. Try to answer each from memory before your exam; every one of them is answerable using Chapters 1–9 of these notes.

10.1 Short Answer (2 Marks)

1. Define client-server architecture.
2. Define P2P architecture.
3. What is a socket?
4. What is the well-known port number for HTTP? For SMTP? For DNS?
5. What is a non-persistent connection?
6. What is a cookie used for?
7. Define DNS.
8. What is a resource record in DNS?
9. What is a torrent?
10. What is a CDN?
11. What is the conditional GET used for?
12. Name the three components of the Internet e-mail system.
13. What is head-of-line blocking?
14. What transport protocol does HTTP always use, and why?

10.2 Medium Answer (5 Marks)

1. Compare client-server and P2P architectures with examples.
2. Explain the four dimensions of transport service that applications may require.
3. Compare TCP and UDP services in detail.
4. Explain non-persistent and persistent HTTP connections with a step-by-step example.
5. Explain the format of an HTTP request message and an HTTP response message.
6. Explain how cookies allow a stateless protocol like HTTP to track users.
7. Explain how Web caching (proxy servers) works and why it is used.
8. Explain the SMTP protocol and how it differs from a mail access protocol like POP3/IMAP.
9. Explain the DNS record types A, NS, CNAME and MX with one example each.
10. Explain how a CDN intercepts and redirects a client's request.
11. Explain how BitTorrent distributes a file among peers, including the role of the tracker.
12. Explain the sequence of socket calls used to build a TCP client-server application.

10.3 Long Answer (10 Marks)

1. Describe the client-server and P2P application architectures in detail, discussing scalability, cost and security trade-offs, with real examples of each.
2. Describe HTTP in detail: overview, non-persistent vs persistent connections, request/response message formats, cookies, and Web caching — illustrate with diagrams.
3. Describe how DNS works: the services it provides, the three-tier server hierarchy (root/TLD/authoritative), DNS caching, and the resource record and message formats.
4. Explain the working of BitTorrent in detail, including torrents, trackers, chunk exchange, rarest-first chunk selection and the tit-for-tat incentive mechanism.
5. Explain how video streaming is delivered at Internet scale: the problems with a single data centre, DASH/adaptive HTTP streaming, and how CDNs (with Netflix/YouTube as examples) solve these problems.
6. Explain socket programming with both UDP and TCP: list every socket-API call needed on the client and server side for each, and explain why the sequences differ.
7. Compare HTTP/1.1 and HTTP/2 in depth, including the head-of-line blocking problem and how HTTP/2's framing sub-layer solves it.



KEY POINT

For every 10-mark question above, aim to include at least one diagram or table — examiners consistently award extra marks for a correctly labelled diagram over pure prose.



EXAM TIP

Good luck for your exam! Work through the Bloom's Question Bank in Chapter 9 once fully, then use this checklist the day before to confirm nothing has slipped.

TMC-203 : Computer Networks

M.C.A. (Second Semester) | Mid Semester Examination, April 2023 | Model Answers

As per exam instructions, each numbered question offers a choice of (a) OR (b), each worth 10 marks. Both options are answered below for complete revision coverage.

Question 1 (a) — 10 Marks (CO1)

Q: What do you mean by computer network? Classify computer networks and various networks criteria in detail.

Answer:

A **computer network** is a collection of two or more interconnected computing devices (computers, servers, printers, mobile devices, etc.) linked together through communication media (wired or wireless) to share data, hardware resources, and applications, and to communicate with one another using a set of defined rules called protocols.

Classification by geographical scope / scale:

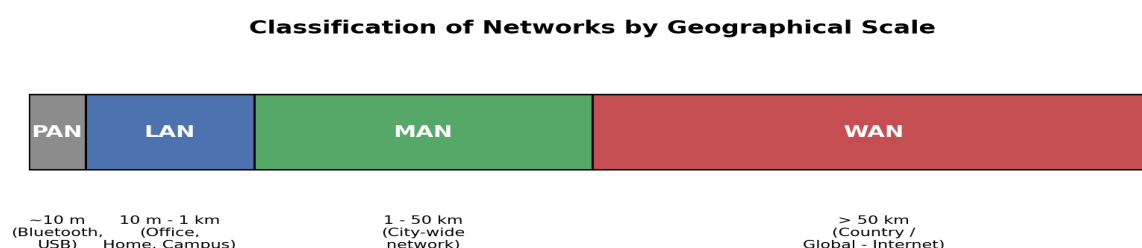


Fig. Classification of networks by geographical scale

- **PAN (Personal Area Network):** covers a very short range (a few metres) around a person — e.g. Bluetooth, USB connections between a phone, laptop and headset.
- **LAN (Local Area Network):** confined to a single building, office, or campus (up to ~1 km); high speed, privately owned — e.g. an office or college network.
- **MAN (Metropolitan Area Network):** spans a city or town (up to ~50 km); often used to interconnect several LANs — e.g. cable TV network, city-wide Wi-Fi.
- **WAN (Wide Area Network):** spans large geographical areas — countries or continents; typically uses leased telecom lines or satellite links — e.g. the Internet.

Other classification criteria:

Criterion	Categories / Description
By Transmission Technology	Broadcast networks: single shared communication channel, message received by all nodes (e.g. traditional Ethernet, wireless LAN). Point-to-Point networks: many individual connections between pairs of nodes; packets may need to pass through several intermediate nodes (e.g. WAN, Internet).
By Topology	Bus, Star, Ring, Mesh, Tree, Hybrid — the physical/logical layout in which nodes are connected.
By Transmission Medium	Wired/Guided: twisted pair, coaxial cable, optical fibre. Wireless/Unguided: radio waves, microwave, infrared, satellite.
By Architecture / Role of Nodes	Client-Server: a central server provides resources/services to client machines. Peer-to-Peer (P2P): all nodes have equal status and can act as both client and server.

Criterion	Categories / Description
By Ownership	Private networks (owned/controlled by a single organisation) vs Public networks (e.g. the Internet, telephone network).
By Switching Technique	Circuit-switched, Message-switched, Packet-switched networks, based on how data is routed through intermediate nodes.

Question 1 (b) — 10 Marks (CO1) (OR option)

Q: What is the purpose of OSI and TCP/IP models in computer networks? How are the OSI model layers segregated by their functions?

Answer:

Both the **OSI model** and the **TCP/IP model** are layered reference frameworks that describe how data communication should occur between two systems over a network. Their purpose is to:

- Break the complex task of network communication into smaller, manageable **layers**, each with a well-defined function.
- Allow **standardization**, so that hardware and software from different vendors can interoperate.
- Enable **independent development** — a change in one layer (e.g. a new cabling technology) does not require changes in other layers.
- Simplify **troubleshooting and teaching** of networking concepts by providing a common reference terminology.

The **OSI model** (ISO, 1984) is a theoretical, 7-layer reference model, while the **TCP/IP model** (used practically on the Internet) is a 4-layer, protocol-based implementation model. The diagram below shows how OSI layers map onto TCP/IP layers.

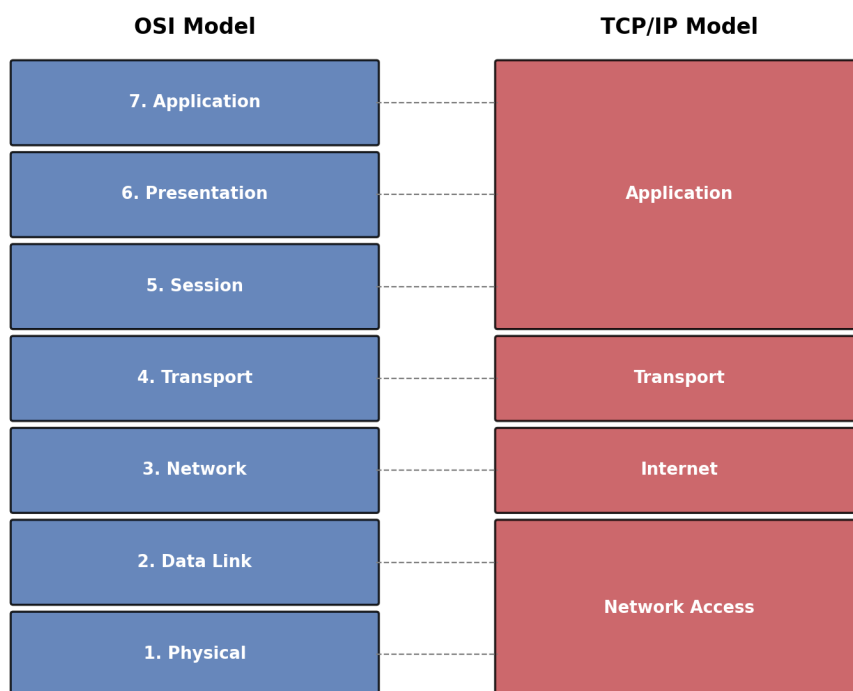


Fig. Mapping of OSI Model layers to TCP/IP Model layers

OSI layers segregated by function:

Layer	Function
7. Application	Interface between network and end-user application software (e.g. browsers, email clients); provides services like file transfer, email, web access.
6. Presentation	Data representation, encryption/decryption, compression, translation between different data formats.
5. Session	Establishes, manages, synchronizes and terminates sessions (dialogues) between two communicating applications.
4. Transport	Provides reliable/unreliable end-to-end delivery, segmentation, flow control and error control (TCP/UDP).

Layer	Function
3. Network	Logical addressing (IP) and routing of packets from source to destination across multiple networks.
2. Data Link	Framing, physical (MAC) addressing, error detection, and media access control on a single link.
1. Physical	Transmission of raw bits over the physical transmission medium (cables, connectors, signal encoding).

Question 2 (a) — 10 Marks (CO2)

Q: How the Email system works? Explain the process of email system with the help of SMTP, MIME, POP and IMAP protocols.

Answer:

Electronic mail (email) allows users to compose, send, and receive messages over a network. The system involves three main components: a **User Agent** (mail client, e.g. Outlook, Gmail app), a **Mail Transfer Agent / Mail Server**, and standard protocols that move mail between them.

Email System: Flow using SMTP, POP3 and IMAP

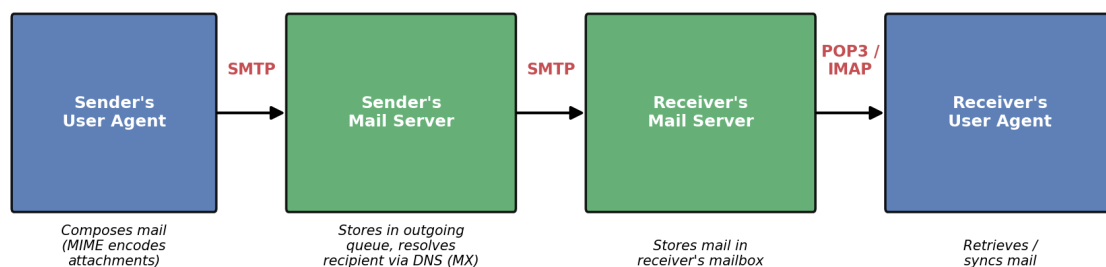


Fig. Flow of an email from sender to receiver

Role of each protocol:

Protocol	Role
SMTP (Simple Mail Transfer Protocol)	Used to send/push mail from the sender's user agent to the sender's mail server, and then relay it between mail servers, until it reaches the receiver's mail server. Uses TCP port 25.
MIME (Multipurpose Internet Mail Extensions)	Extends the original text-only email format to allow attachments — images, audio, video and other binary/non-ASCII data — by encoding them and describing content-type headers.
POP3 (Post Office Protocol v3)	Used by the receiver's user agent to download/pull mail from the mail server to the local device; by default mail is deleted from the server after download (offline access).
IMAP (Internet Message Access Protocol)	Allows the user agent to manage mail directly on the server — read, organize into folders, search — while keeping a synchronized copy on the server, enabling access from multiple devices.

Step-by-step process:

- Sender composes a message in the User Agent; MIME encodes any attachments.
- The message is transferred using **SMTP** from the sender's UA to the sender's mail server, and then relayed to the receiver's mail server.
- The receiver's mail server stores the message in the recipient's mailbox.
- The receiver's User Agent retrieves the mail using **POP3** (download and typically delete from server) or **IMAP** (synchronize while keeping mail on server).

Question 2 (b) — 10 Marks (CO2) (OR option)

Q: Why HTTP protocol is used in application layer? Discuss the Non-persistent and Persistent connections of HTTP protocol in detail.

Answer:

HTTP (HyperText Transfer Protocol) is an application-layer protocol used for transferring web resources — HTML pages, images, videos, etc. — between a web server and a web client (browser). It resides at the application layer because:

- It directly defines rules for how **application software** (browsers and web servers) should request and respond with data — not concerned with routing or physical transmission.
- It uses the reliable, connection-oriented transport service of **TCP** (usually port 80) beneath it, so it can focus purely on the request/response semantics.
- It is a **stateless, request-response protocol** using simple text-based methods (GET, POST, PUT, DELETE) that only application programs need to understand.

Non-persistent vs Persistent HTTP connections:

Aspect	Non-Persistent Connection	Persistent Connection
Definition	A new TCP connection is opened for each object requested (e.g. HTML file, each image), and closed after that object is delivered.	A single TCP connection remains open and is reused to transfer multiple objects/requests between client and server.
Default in	HTTP/1.0	HTTP/1.1 (and later) — default behaviour
Overhead	High — each object requires a new TCP 3-way handshake (connection setup + slow start), increasing delay.	Low — connection is set up once; subsequent requests avoid repeated handshake overhead.
Performance	Slower for pages with many embedded objects, since multiple RTTs are needed.	Faster, especially for pages with many small objects, since RTT is saved for each additional request.
Parallelism	Browsers often open multiple parallel non-persistent connections to fetch objects simultaneously.	Requests can be pipelined over the same connection (with or without waiting for prior responses, depending on mode).

In short, HTTP is placed at the application layer because it defines how web applications exchange data, while persistent connections were introduced in HTTP/1.1 to reduce the connection-setup overhead of the original non-persistent design, improving page-load performance.

Question 3 (a) — 10 Marks (CO1)

Q: What do you mean by topologies in computer networks? Explain the different types of topologies based on the following points: (i) Number of cables required (ii) Number of ports required (iii) Advantages and disadvantages

Answer:

A **network topology** is the physical or logical arrangement in which the nodes (computers, devices) of a network are connected to each other. It determines cost, performance, reliability, and ease of troubleshooting of the network.

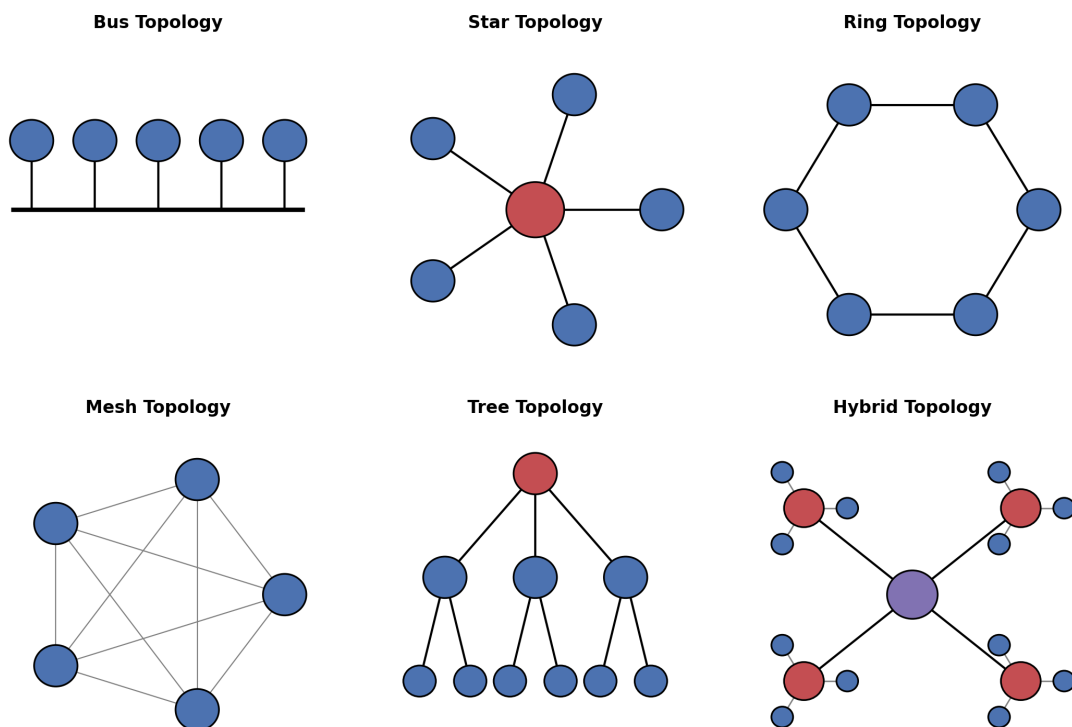


Fig. Common network topologies

Topology	Cables Required (for n nodes)	Ports Required per Node	Advantages	Disadvantages
Bus	1 backbone cable (+ drop lines)	1 port (tap onto bus)	Cheap, less cabling, easy for small networks	Entire network fails if backbone breaks; performance drops as nodes increase
Star	n (one per node to hub/switch)	1 port per node (+ n ports on hub)	Easy to install/manage; failure of one link doesn't affect others	If central hub/switch fails, whole network is down; more cable needed
Ring	n (each node to 2 neighbours)	2 ports per node	Orderly, collision-free access (token passing); easy to install	Break in ring can disrupt whole network (unless dual ring); hard to troubleshoot
Mesh (full)	$n(n-1)/2$	n-1 ports per node	Highly reliable, fault-tolerant, secure dedicated links	Very expensive; complex cabling and maintenance
Tree	n-1 (hierarchical, like extended star)	Varies by level (1 to several)	Scalable, hierarchical management, easy fault isolation	Backbone/root failure affects all dependent branches
Hybrid	Depends on combination used	Depends on combination used	Flexible, combines strengths of constituent topologies	Complex design; higher cost of hardware and maintenance

Question 3 (b) — 10 Marks (CO1) (OR option)

Q: What is the significance of switching? Explain the various switching techniques with a neat and clean diagram.

Answer:

In a network with many nodes, it is impractical to have a dedicated physical link between every pair of nodes. **Switching** is the technique/mechanism used by intermediate nodes (switches) to forward data from a source to a destination through a series of intermediate nodes efficiently, without needing a direct physical connection between every pair of communicating devices. Its significance lies in enabling **efficient resource sharing** of communication links, **scalability** to large numbers of nodes, and **flexible routing** of data across the network.

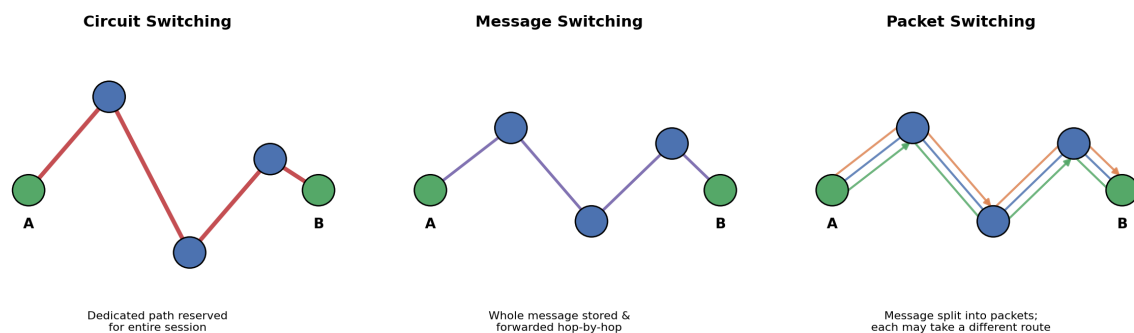


Fig. Circuit, Message and Packet Switching

1. Circuit Switching

A dedicated physical path (circuit) is established between sender and receiver before communication begins, and is reserved for the entire duration of the session (e.g. traditional telephone networks). It occurs in three phases: **connection establishment, data transfer, connection termination**.

- Advantage: guaranteed, constant bandwidth and no delay once set up.
- Disadvantage: inefficient use of bandwidth if the link is idle; connection setup takes time.

2. Message Switching

The entire message is sent as a single unit from one node to the next (store-and-forward); each intermediate node stores the whole message and forwards it once the outgoing link is free.

- Advantage: no dedicated path needed; efficient use of bandwidth.
- Disadvantage: high storage requirement at nodes; not suitable for real-time traffic due to delay.

3. Packet Switching

The message is broken into smaller units called **packets**, each carrying source/destination addresses, which are routed independently (and may take different paths) through the network, and reassembled at the destination. This is the switching technique used in the Internet.

- Advantage: efficient bandwidth utilization, robust (can route around failures), supports many users simultaneously.
- Disadvantage: packets may arrive out of order or be delayed (variable latency/jitter); needs reassembly logic.

Question 4 (a) — 10 Marks (CO2)

Q: What is the use of Domain Name System (DNS)? Explain the Domain Name, Name spaces and Name-address resolution techniques in detail.

Answer:

The **Domain Name System (DNS)** is a hierarchical, distributed naming system that translates human-readable domain names (e.g. `www.google.com`) into numeric IP addresses (e.g. `142.250.190.14`) that computers use to identify each other on a network. Its main uses are:

- Allows users to remember easy-to-use **names** instead of numeric IP addresses.
- Provides a **distributed database** so no single server needs to store the entire mapping, improving scalability and fault tolerance.
- Supports load balancing (mapping one name to multiple IPs) and mail routing (via MX records).

Domain Name and Name Spaces:

A **domain name** is a sequence of labels separated by dots (e.g. `mail.google.com`), read from the most specific (leftmost) to the most general (rightmost/root) label. The **name space** is the entire set of possible names, organized as an inverted tree structure with a **root** at the top:

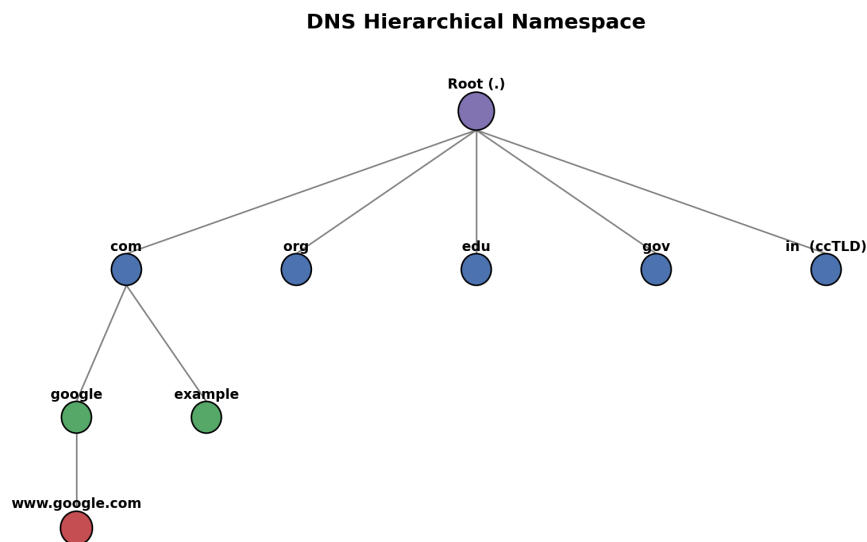


Fig. DNS hierarchical name space

- **Root level:** represented by a single dot "."; contains references to all Top-Level Domain servers.
- **Top-Level Domain (TLD):** generic (.com, .org, .edu, .gov) or country-code (.in, .uk) domains.
- **Second-Level Domain:** the organisation's registered name (e.g. `google` in `google.com`).
- **Sub-domain / Host:** further labels identifying a specific host or service (e.g. `www`, `mail`).

There are two name-space structuring styles: a **flat name space** (a single unstructured list of names, not scalable) and a **hierarchical name space** (as used by DNS, where each name is defined with respect to its position in a tree, allowing decentralised management).

Name-Address Resolution Techniques:

Resolution is the process of mapping a name to its corresponding address, performed by a resolver (client software) querying DNS servers. Two common techniques:

- **Recursive Resolution:** the client sends a query to a local DNS (resolver) server, and that server takes full responsibility for contacting root, TLD and authoritative servers on the client's behalf, finally returning the complete answer to the client.
- **Iterative Resolution:** the DNS server, instead of doing the whole lookup itself, gives back the best answer it can (often a referral to another server), and the client (or local resolver) repeats the query to the next server itself, until it reaches the authoritative server holding the answer.

In practice, the client typically performs a recursive query to its local DNS resolver, while that resolver performs iterative queries to root, TLD and authoritative servers on the client's behalf; results are also cached at each level (with a Time-To-Live) to speed up future lookups.

Question 4 (b) — 10 Marks (CO2) (OR option)

Q: Discuss the principles and architectures of network applications in detail.

Answer:

A **network application** is software that runs on multiple end systems (hosts) and communicates over the network to accomplish a task (e.g. web browsing, email, video streaming, file sharing). Application developers design the application to run on end systems only — there is no need to write software for network-core devices such as routers, since the core simply forwards data (this principle is known as the **end-to-end principle**, which keeps the network core simple and pushes complexity/intelligence to the edge).

Key principles of network application design:

- **Processes communicate via sockets:** a process sends/receives messages through a software interface called a **socket**, which acts as the boundary between the application layer and the transport layer.
- **Addressing:** a process is identified by the combination of an **IP address** (identifying the host) and a **port number** (identifying the specific process on that host).
- **Choice of transport service:** depending on requirements — reliable data transfer, throughput, timing/delay, security — the application chooses an appropriate transport protocol (e.g. TCP for reliability, UDP for speed).
- **Application-layer protocol:** defines the types of messages exchanged (request/response, etc.), message syntax, field meanings, and rules for when/how messages are sent (e.g. HTTP, SMTP, FTP).

Architectures of network applications:

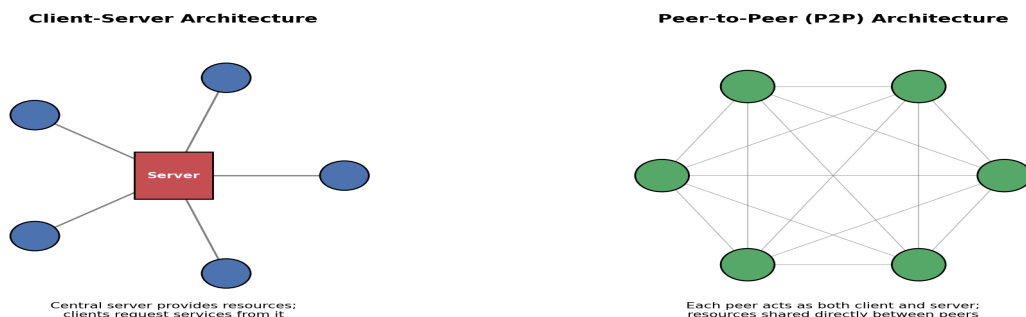


Fig. Client-Server vs Peer-to-Peer (P2P) architecture

Architecture	Description	Examples
Client-Server	A single, always-on server host provides services/resources to many client hosts, which initiate contact with the server. Clients do not directly communicate with each other. The server typically has a fixed, well-known IP address and needs high capacity to serve many clients (often deployed as data centers).	Web (HTTP), Email (SMTP), FTP
Peer-to-Peer (P2P)	There is minimal (or no) reliance on always-on servers; instead, pairs of intermittently-connected hosts, called peers , communicate directly with each other. Peers are not owned by a service provider, and each peer can act as both client and server (self-scalability — more peers means more service capacity).	BitTorrent file sharing, Skype (originally), blockchain networks
Hybrid (Client-Server + P2P)	Combines both models — e.g. a central server is used for discovery/coordination (such as finding peers), while the actual data transfer happens directly, peer-to-peer.	BitTorrent (uses trackers), some instant messaging systems

Question 5 (a) — 10 Marks (CO1)

Q: What is data communication system in computer networks? Discuss the data communication components, characteristics and modes of data flow in detail.

Answer:

A **data communication system** is the exchange of data (in the form of 0s and 1s) between two devices through some form of transmission medium, following a set of agreed rules (protocol), such that the information is delivered accurately and efficiently from a source to a destination.

Components of a Data Communication System

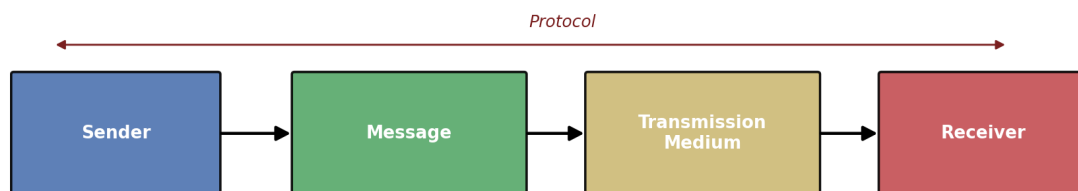


Fig. Components of a data communication system

Components of Data Communication:

Component	Description
Message	The information (data) to be communicated — can be text, numbers, images, audio, or video.
Sender	The device that sends the data message — e.g. a computer, workstation, telephone handset.
Receiver	The device that receives the message — e.g. computer, telephone, printer.
Medium (Transmission Medium)	The physical path through which the message travels from sender to receiver — e.g. twisted-pair cable, coaxial cable, fibre-optic cable, or radio waves.
Protocol	A set of rules that governs data communication, representing an agreement between the communicating devices; without a common protocol, two devices may be connected but not communicating.

Characteristics of effective data communication:

- **Delivery:** data must be delivered to the correct destination only.
- **Accuracy:** data must be delivered accurately, without alteration.
- **Timeliness:** data must be delivered in a timely manner; late delivery (e.g. of audio/video) is useless.
- **Jitter:** uneven delay in the delivery of packets should be minimized, especially for audio/video data.

Modes of Data Flow (Transmission Modes):

Mode	Description	Example
Simplex	Communication is unidirectional — only one of the two devices can transmit; the other can only receive.	Keyboard to CPU, Television broadcasting
Half-Duplex	Each device can both send and receive , but not at the same time — when one sends, the other only receives, and vice versa.	Walkie-talkies
Full-Duplex	Both devices can send and receive simultaneously ; the channel capacity is shared or doubled for both directions.	Telephone network

Question 5 (b) — 10 Marks (CO2) (OR option)

Q: What is p2p file distribution in computer networks? Explain the working of Bit Torrent in detail.

Answer:

P2P (peer-to-peer) file distribution is a method of distributing files across a network in which there is no reliance on a central server to store and serve the entire file to every requester. Instead, each participating host (**peer**) that downloads part of the file also simultaneously uploads (shares) the parts it already has to other peers, so that peers collectively both consume and contribute bandwidth and storage. This makes P2P distribution highly **self-scalable** — as more peers join wanting the file, more upload capacity is also added to the system.

Working of BitTorrent:

BitTorrent is one of the most widely used P2P file-distribution protocols. Its key elements and working are:

- **Torrent / Swarm:** the collection of all peers participating in the distribution of a particular file is called a **torrent** (or swarm).
- **File splitting into chunks:** the file being distributed is divided into fixed-size **chunks** (pieces), typically 256 KB each.
- **Tracker:** when a peer joins the torrent, it registers with a **tracker** — an infrastructure node that keeps track of the peers participating in that torrent, and provides the newly joined peer with a list of other participating peers to connect to.
- **Chunk exchange:** a joining peer initially has no chunks, but accumulates more chunks over time by downloading them from other peers. While downloading, it also uploads chunks it has already obtained to other peers, so it also acts as a server.
- **Requesting missing chunks:** a peer periodically asks each of its neighbouring peers for the list of chunks they have, and requests the chunks it is **missing**. A common strategy is "**rarest first**" — requesting the chunks that are least common among neighbours first, so those rare chunks get replicated quickly and remain available in the swarm.
- **Choking / Tit-for-tat incentive:** to encourage peers to keep uploading (rather than only downloading, i.e. "free-riding"), a peer typically sends chunks to a limited number of neighbours that are currently supplying it data at the highest rate (this is called "**tit-for-tat**"). Peers not among these top uploaders are periodically "**choked**" (temporarily denied chunks), which incentivizes all peers to keep contributing upload bandwidth.
- **Completion:** once a peer has downloaded all the chunks of the file, it can leave the swarm, or optionally remain to continue uploading ("seeding") to help other peers still downloading.

Because every downloading peer also uploads to others, BitTorrent distributes the bandwidth load across the whole swarm instead of overloading a single server, allowing very large files to be shared efficiently among a large number of users.

TMC-203 : Computer Networks

M.C.A. (Second Semester) | Mid Semester Examination, 2021-22 | Model Answers

Question 1 (a) — 10 Marks (CO1)

Q: Explain the OSI model with the discussion of the services at each layer, devices, and address with proper diagrams.

Answer:

The **OSI (Open Systems Interconnection) Reference Model**, developed by ISO in 1984, is a conceptual framework that standardizes the functions of a communication system into **seven layers**. Each layer performs a specific set of services and communicates only with the layer directly above and below it, and with the same layer on the remote system (peer-to-peer communication) via a defined protocol.

The OSI Reference Model (7 Layers)

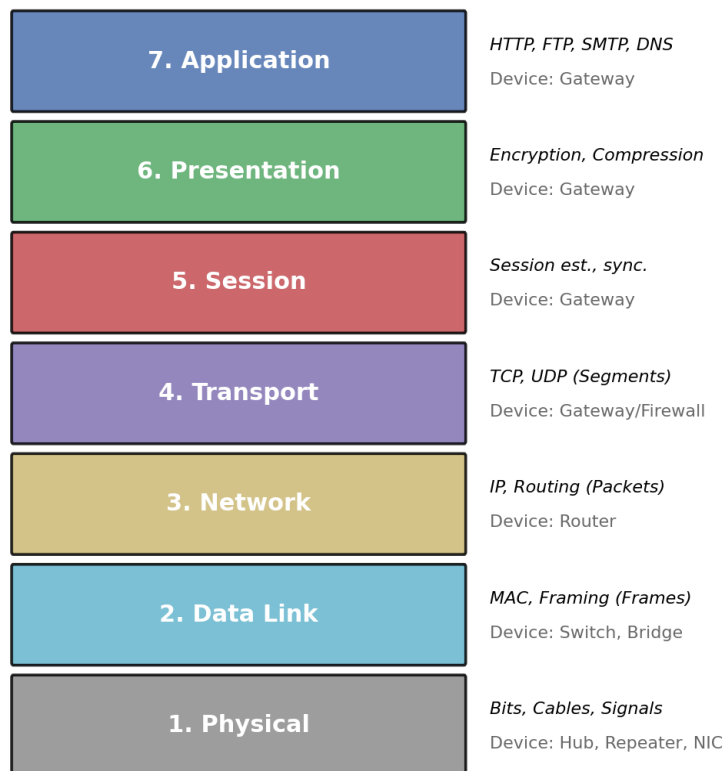


Fig 1.1 — The Seven Layers of the OSI Model

Layer	Main Function / Services	Data Unit	Typical Devices	Addressing
7. Application	Provides network services directly to end-user applications (email, file transfer, web browsing)	Data / Message	Gateway	N/A (service ports, e.g. HTTP:80)
6. Presentation	Data translation, encryption/decryption, compression, character-set conversion (ASCII/EBCDIC)	Data	Gateway	N/A
5. Session	Establishes, manages and terminates sessions between two communicating hosts; synchronization (checkpoints)	Data	Gateway	N/A

Layer	Main Function / Services	Data Unit	Typical Devices	Addressing
4. Transport	End-to-end delivery, segmentation & reassembly, error control, flow control (TCP = reliable, UDP = connectionless)	Segment	Firewall, Gateway	Port Address (16-bit port no.)
3. Network	Logical addressing and routing of packets across multiple networks; path determination	Packet	Router, Layer-3 Switch	Logical / IP Address
2. Data Link	Framing, physical (MAC) addressing, error detection (CRC), flow control on a single link; divided into LLC & MAC sub-layers	Frame	Switch, Bridge, NIC	Physical / MAC Address
1. Physical	Transmission of raw bits over the physical medium; defines voltage levels, cabling, connectors, data rate, topology	Bit	Hub, Repeater, Cables, Connectors	N/A

Key points on addressing at each layer:

- **Physical layer:** deals only with raw bits; no address is used at this layer.
- **Data Link layer:** uses the **MAC (Media Access Control) address** — a 48-bit physical address burned into the NIC, used for communication within the same local network.
- **Network layer:** uses the **logical (IP) address** — a 32-bit (IPv4) or 128-bit (IPv6) address used to route packets across different networks.
- **Transport layer:** uses the **port address** (16-bit) to identify the specific process/application on the host (e.g. port 80 for HTTP).
- **Upper layers (Session, Presentation, Application):** generally use logical, application-specific names such as URLs or email addresses rather than a fixed numeric address.

In short, data generated by an application at the sender travels down the stack, where each layer adds its own header (encapsulation) — forming segments, then packets, then frames, then bits — and at the receiver the reverse process (decapsulation) removes each header as the data climbs back up the stack.

Question 1 (b) — 10 Marks (CO1) (Alternative / OR option to Q1(a))

Q: Explain the various types of topologies with their advantages and disadvantages with neat and clean diagram.

Answer:

Network topology refers to the physical or logical arrangement in which nodes (computers, devices) are connected in a network. The choice of topology affects cost, reliability, scalability, and ease of troubleshooting. The main types are described below.

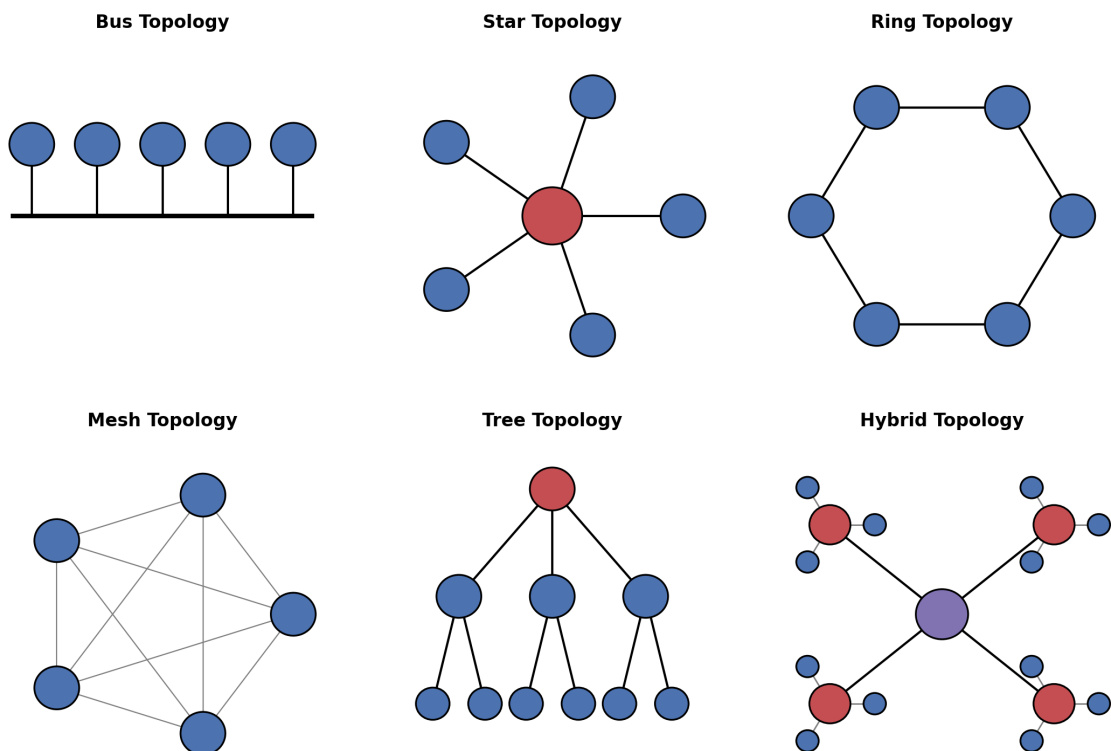


Fig 1.2 — Common Network Topologies

1. Bus Topology

All devices are connected to a single central cable, called the **backbone** or bus. Data sent by any node travels along the bus and is received by all nodes, but accepted only by the intended recipient.

Advantages:

- Easy and inexpensive to install for small networks.
- Requires less cable than other topologies.

Disadvantages:

- Entire network fails if the main cable breaks.
- Performance degrades as more nodes are added (collisions).
- Difficult to troubleshoot faults.

2. Star Topology

Every node is connected individually to a **central device** (hub or switch). All communication between nodes passes through this central device.

Advantages:

- Easy to install, manage and troubleshoot; failure of one node/cable does not affect others.
- Easy to add or remove nodes without disturbing the network.

Disadvantages:

- If the central hub/switch fails, the entire network goes down.
- Requires more cable than bus topology.
- Cost depends on the central device.

3. Ring Topology

Each node is connected to exactly two neighbouring nodes, forming a closed loop (ring). Data travels around the ring in one direction (or both, in a dual ring) until it reaches the destination.

Advantages:

- Data flow is orderly and collision-free (using token passing).
- Easy to install and reconfigure for small networks.

Disadvantages:

- Failure of a single node/cable can break the entire ring (unless a dual ring is used).
- Adding/removing a node disrupts network activity.
- Troubleshooting is difficult.

4. Mesh Topology

Every node is connected directly to every other node, providing multiple paths for data to travel. Can be a **full mesh** (all nodes interconnected) or **partial mesh**.

Advantages:

- Highly reliable — failure of one link does not isolate any node.
- High fault tolerance and data security (dedicated links).

Disadvantages:

- Very expensive and complex due to the large amount of cabling.
- Difficult to install and maintain as the number of nodes grows ($n(n-1)/2$ links).

5. Tree Topology

A hierarchical topology that combines characteristics of star and bus topologies — groups of star-configured nodes are connected to a linear bus backbone, forming a tree/hierarchical structure with a root node.

Advantages:

- Easily scalable and manageable in a hierarchical manner.
- Fault isolation is easier; a problem in one branch does not affect others.

Disadvantages:

- If the root/backbone fails, the connected branches lose communication.
- Cabling and maintenance cost increases with size.

6. Hybrid Topology

A combination of two or more different basic topologies (e.g. star-bus, star-ring) to meet the specific needs of an organization, drawing on the strengths of each.

Advantages:

- Flexible, reliable, and scalable — combines the advantages of the topologies used.
- Faults can often be isolated to a smaller segment.

Disadvantages:

- Complex design and higher implementation/maintenance cost.
- Requires careful planning and more hardware (hubs, switches, bridges).

A Note on Questions 2 – 5

In the uploaded scan, **page 2 of the question paper (containing Questions 2, 3, 4 and 5) did not scan properly** — only the question numbers and (a)/(b) labels are faintly visible; the actual question text is missing/blank on that page. I tried enhancing contrast and brightness on the image but there is no recoverable text underneath.

To complete this answer set, please do one of the following and share it with me:

- Re-scan/re-photograph page 2 of the question paper in better lighting/focus, or
- Type out Questions 2, 3, 4, and 5 (and their (a)/(b) options) as text.

Once I have the question text, I will add fully worked model answers for Questions 2–5 to this same document, organized by marks just like Question 1 above.

Graphic Era Hill University

Mid Term (Odd) Semester Examination — October 2024

Computer Networks (TMC 102)

Model Answer Key — MCA 1st Semester

This document provides detailed, exam-ready answers for **both sub-questions (a) and (b)** of every question in the mid-term paper, so you can revise the complete syllabus rather than only the options you might choose in the exam. Each answer includes definitions, diagrams described in words, comparison tables, and examples to help you write full 10-mark answers.

Course	MCA — 1st Semester
Paper	Computer Networks (TMC 102)
Max Marks	50 (5 Questions × 10 Marks)
Instructions followed	Each Q has two options (a)/(b); both are answered here for complete coverage

Question 1 (10 Marks)

(a). Define data communication. What are the essential components involved in data communication?

[CO1]

Data communication refers to the process of exchanging data or information between two or more devices through some form of transmission medium such as a wire cable, optical fibre, or wireless link. For communication to be meaningful, the data being transmitted must be delivered accurately, in the correct order, and in a timely manner, and it must be understood correctly by the receiving device. The effectiveness of a data communication system depends on four fundamental characteristics: **delivery** (data must reach the intended device and no other), **accuracy** (data must be delivered without errors), **timeliness** (data must be delivered without significant delay, especially for real-time applications such as audio/video), and **jitter** (the variation in packet arrival time must be minimal for smooth playback of real-time data).

Essential components of data communication: A data communication system is made up of five basic components that work together to move data from a source to a destination.

- **Message:** The information (data) to be communicated. It can be in the form of text, numbers, images, audio, or video.
- **Sender:** The device that sends the message. It can be a computer, workstation, telephone, or video camera.
- **Receiver:** The device that receives the message. It can also be a computer, workstation, telephone, or television.
- **Medium (Transmission Medium):** The physical path through which the message travels from sender to receiver. It can be wired (twisted pair, coaxial cable, fibre optic) or wireless (radio waves, microwaves, infrared).
- **Protocol:** A set of rules that governs data communication. It represents an agreement between the communicating devices about how the data is formatted, transmitted, and interpreted. Without a protocol, two devices may be connected but not communicating, just as a person speaking French cannot be understood by someone who understands only Japanese.

Together, these five components form the basic model of data communication: the sender encodes the message and transmits it over the medium according to an agreed protocol, and the receiver decodes it back into usable information.

Exam tip: Draw a simple block diagram — Sender → Medium → Receiver, with 'Protocol' labelled above and 'Message' as the arrow content — for full marks.

(b). Explain the client-server architecture. How does it differ from a peer-to-peer architecture?

[CO2]

Client-Server Architecture is a network model in which the workload is divided between two types of programs/systems: **clients**, which request services or resources, and **servers**, which provide (serve) those services or resources. The server is typically a powerful, centralized machine (e.g., a web server, database server, or mail server) that runs continuously and waits for requests, while clients are the end-user machines (e.g., a browser on a laptop or a mobile app) that initiate requests whenever a user needs a service.

Working: A client sends a request message to the server over the network using an agreed protocol (e.g., HTTP for the web, SMTP for mail). The server processes the request, retrieves or computes the required data, and returns a response back to the client. A single server can typically service many clients simultaneously. Common examples include: web browsing (browser = client, web server = server), email (mail client = client, mail server = server), and online banking.

Peer-to-Peer (P2P) Architecture is a network model in which all connected devices, called **peers**, have equal status and capability — every peer can act as both a client and a server at the same time. There is no central server dedicated to providing services; instead, resources (files, processing power, storage) are shared directly between peers. Examples include BitTorrent file sharing and some blockchain networks.

Basis	Client-Server Architecture	Peer-to-Peer Architecture
Role of nodes	Fixed roles: dedicated server(s) and multiple clients	Equal roles: every peer can be both client and server
Centralization	Centralized — server manages resources and access	Decentralized — no central authority
Scalability	Limited by server capacity; needs upgrades/load-balancing	Scales well as new peers add their own resources
Cost	Higher (dedicated server hardware & maintenance)	Lower (uses existing peer resources)
Reliability / Single point of failure	Server failure can bring down the whole service	More fault-tolerant; failure of one peer has limited effect
Security & management	Easier to secure and administer centrally	Harder to manage/secure — no central control
Examples	Web servers, email servers, database servers	BitTorrent, Skype (early versions), blockchain networks

Exam tip: If asked to 'explain', always describe the working first, then use a comparison table — examiners award marks for structure.

Question 2 (10 Marks)

(a). Compare and contrast various network topologies (star, ring, bus, mesh) in terms of advantages and disadvantages.

[CO1]

Network topology refers to the physical or logical arrangement of nodes (computers, switches, etc.) and links in a network.

1. Bus Topology: All devices are connected to a single central cable, called the bus or backbone. Data sent by one node travels along the bus and is received by all other nodes, but only the intended recipient accepts it.

- **Advantages:** Simple to install; requires less cabling, hence cost-effective for small networks; easy to extend.

- **Disadvantages:** Entire network fails if the main cable breaks; performance degrades as more devices are added (collisions increase); difficult to troubleshoot; limited cable length and number of nodes.

2. Star Topology: All devices are connected individually to a central device such as a hub or switch. All data passes through this central node.

- **Advantages:** Failure of one cable/node does not affect others; easy to install, manage, and troubleshoot; good performance since collisions are minimised (especially with a switch); easy to add or remove devices.

- **Disadvantages:** If the central hub/switch fails, the entire network goes down; requires more cabling than bus; cost depends on the central device.

3. Ring Topology: Each device is connected to exactly two other devices, forming a circular path for data, which travels in one direction (or both, in a dual ring).

- **Advantages:** Data flows in an orderly, predictable manner; performs better than bus under heavy load since each node gets a chance to transmit (token passing); no collisions in token-ring systems.

- **Disadvantages:** Failure of a single node or cable can disrupt the entire ring (unless a dual ring is used); adding/removing a node requires temporarily breaking the ring; troubleshooting is difficult.

4. Mesh Topology: Every device is connected directly to every other device (full mesh) or to several other devices (partial mesh), providing multiple paths for data.

- **Advantages:** Highly reliable and fault-tolerant — failure of one link does not isolate any node since alternate paths exist; no traffic congestion since dedicated links are used; enhanced privacy and security since data travels on a dedicated line.

- **Disadvantages:** Very expensive and complex due to the large amount of cabling and I/O ports required ($n(n-1)/2$ links for n nodes in a full mesh); difficult to install and reconfigure; wastes bandwidth if links are idle.

Topology	Cabling Cost	Fault Tolerance	Scalability	Typical Use
Bus	Low	Poor (single cable failure = network down)	Poor	Small, legacy LANs
Star	Medium	Good (isolated failures) but hub is SPOF	Good	Most modern LANs (Ethernet)
Ring	Medium	Moderate (single break can disrupt ring)	Moderate	Token Ring, FDDI networks

Topology	Cabling Cost	Fault Tolerance	Scalability	Typical Use
Mesh	Very High	Excellent (multiple redundant paths)	Poor (cost grows quickly)	Backbone/critical networks (e.g., WAN cores)

Exam tip: Sketch each topology (dot-and-line diagrams) alongside the text — diagrams fetch extra marks in networking papers.

(b). What is the format of an HTTP message? Describe the components of both request and response messages.

[CO2]

HTTP (HyperText Transfer Protocol) is an application-layer protocol used for communication between web browsers (clients) and web servers. It works on a request-response model and is a text-based protocol, meaning messages are human-readable. There are two types of HTTP messages: **request messages** (sent by the client) and **response messages** (sent by the server).

1. HTTP Request Message Format consists of the following parts:

- **Request Line:** Contains the HTTP method (e.g., GET, POST, PUT, DELETE), the requested URL/resource path, and the HTTP version. Example: *GET /index.html HTTP/1.1*
- **Header Lines:** Provide additional information about the request, such as Host, User-Agent, Accept, Accept-Language, Connection, Content-Type, and Content-Length. Each header is written as 'Name: Value'.
- **Blank Line:** A blank line (CRLF) marks the end of the headers and separates them from the body.
- **Body (optional):** Contains data sent to the server, used mainly with POST/PUT requests (e.g., form data or JSON payload). GET requests typically have no body.

2. HTTP Response Message Format consists of:

- **Status Line:** Contains the HTTP version, a numeric status code, and a reason phrase. Example: *HTTP/1.1 200 OK*. Common status codes: 200 (OK), 301 (Moved Permanently), 400 (Bad Request), 404 (Not Found), 500 (Internal Server Error).
- **Header Lines:** Provide metadata about the response, such as Content-Type, Content-Length, Date, Server, Set-Cookie, and Cache-Control.
- **Blank Line:** Separates the headers from the body, exactly as in the request.
- **Body:** Contains the actual resource requested — e.g., the HTML page, image, JSON data, or file content.

Field	Request Message	Response Message
First line	Request line: Method + URL + Version	Status line: Version + Status code + Reason phrase
Example	GET /home.html HTTP/1.1	HTTP/1.1 200 OK
Headers	Host, User-Agent, Accept, Cookie	Content-Type, Content-Length, Server, Set-Cookie
Body	Optional (form/JSON data in POST/PUT)	Usually present (the requested resource)

Exam tip: Write out one complete sample GET request and its 200 OK response line-by-line — this is a favourite way to score full marks on HTTP format questions.

Question 3 (10 Marks)

(a). Compare the OSI model with the TCP/IP protocol stack. What are the key differences?

[CO1]

The **OSI (Open Systems Interconnection) Model** is a conceptual, 7-layer reference model developed by ISO that describes how data communication should occur between two systems. It was designed as a theoretical, protocol-independent framework. The **TCP/IP Model** is a practical, 4-layer (sometimes described as 5-layer) protocol suite that is actually implemented and used to run the modern Internet.

OSI Model Layers (top to bottom): Application, Presentation, Session, Transport, Network, Data Link, Physical.

TCP/IP Model Layers (top to bottom): Application, Transport, Internet, Network Access (Link/Physical) — the Application layer of TCP/IP combines the OSI's Application, Presentation, and Session layers, and the Network Access layer combines OSI's Data Link and Physical layers.

Basis	OSI Model	TCP/IP Model
Full form	Open Systems Interconnection	Transmission Control Protocol / Internet Protocol
Number of layers	7 layers	4 layers (Application, Transport, Internet, Network Access)
Development approach	Layers defined first, then protocols were fitted to them (theoretical)	Protocols were developed first, model was described afterwards (practical)
Usage	Used mainly as a reference/teaching model	Used as the actual protocol suite running the Internet
Layer independence	Strict separation; each layer clearly defined	Layers are more tightly coupled (e.g., Application, Session, Presentation merged)
Reliability	Both connection-oriented and connectionless services supported at Network layer	Network (Internet) layer is connectionless (IP); reliability handled at Transport layer (TCP)
Protocol examples	Not protocol specific — generic model	Concrete protocols: IP, TCP, UDP, HTTP, FTP, DNS, SMTP
Approach	Vertical, more rigid, layer-by-layer approach	Horizontal, more flexible, protocol-driven approach

Key differences summarised:

- OSI has **7 layers**; TCP/IP has **4 layers**.
- OSI is a **generic, protocol-independent** model; TCP/IP is **protocol-specific** and is actually implemented.
- OSI separates Session and Presentation layers; TCP/IP merges their functions into the Application layer.
- OSI was developed before its protocols existed; TCP/IP protocols existed first and the model was derived afterward.

- TCP/IP is the model actually used on the Internet today, while OSI mainly serves as a teaching/reference tool.

Exam tip: Draw both models side by side with arrows showing which OSI layers map onto which TCP/IP layer — this visual is a classic 10-mark answer.

(b). Describe the File Transfer Protocol (FTP). What are its primary commands, and how does it differ from HTTP?

[CO2]

FTP (File Transfer Protocol) is an application-layer protocol used to transfer files between a client and a server over a TCP/IP network (typically the Internet). It runs on top of TCP and uses **two separate connections**: a **control connection** (port 21) used to send commands and receive responses, which remains open for the duration of the session, and a **data connection** (port 20, or a negotiated port in passive mode) used only to actually transfer the file data, opened separately for each file transfer and closed once the transfer is done.

FTP supports two modes of operation: **Active mode**, where the server initiates the data connection back to the client, and **Passive mode**, where the client initiates both connections (commonly used to work around firewalls/NAT). FTP can also work in ASCII mode (for text files) or Binary mode (for non-text files such as images or executables).

Primary FTP commands:

- **USER** — specifies the username for login
- **PASS** — specifies the password for the given username
- **LIST / LS** — lists files and directories on the server
- **RETR** (retrieve) — downloads a file from the server to the client
- **STOR** (store) — uploads a file from the client to the server
- **DELE** — deletes a specified file on the server
- **CWD** (change working directory) — changes the current directory on the server
- **PWD** — prints the current working directory
- **MKD / RMD** — creates/removes a directory on the server
- **QUIT** — terminates the FTP session

Basis	FTP	HTTP
Purpose	Designed specifically for transferring files (upload/download)	Designed for transferring hypertext/web pages and general resources
Connections	Uses two separate connections — control (port 21) and data (port 20/negotiated)	Uses a single connection for both request and response
State	Stateful — maintains session state (login, current directory) across commands	Stateless — each request is independent (state added via cookies/sessions if needed)
Data types	Supports ASCII and binary transfer modes explicitly	Uses MIME/Content-Type headers to describe data type

Basis	FTP	HTTP
Authentication	Built-in login (USER/PASS commands), or anonymous login	No built-in login; uses separate mechanisms (Basic Auth, tokens, cookies)
Typical use	Bulk file transfer between systems	Browsing websites, APIs, retrieving/posting web resources

Exam tip: Mention the two-connection (control + data) architecture explicitly — it's the single most-tested FTP concept.

Question 4 (10 Marks)

(a). Compare and contrast circuit switching and packet switching. What are the benefits and drawbacks of each?

[CO1]

Switching refers to the method used to establish a connection and transfer data between a sender and receiver across a network. The two major switching techniques are circuit switching and packet switching.

Circuit Switching: A dedicated communication path (circuit) is established between the sender and receiver *before* data transfer begins, and this path remains reserved exclusively for that connection until the session ends. The traditional telephone network (PSTN) is the classic example, where a fixed circuit is set up for the duration of a call. It involves three phases: **circuit establishment**, **data transfer**, and **circuit disconnection**.

- **Benefits:** Guaranteed, dedicated bandwidth for the entire session; consistent, predictable transmission delay (no variable queuing); data arrives in order since it follows a fixed path; well-suited to continuous, real-time traffic such as voice calls.
- **Drawbacks:** Inefficient use of bandwidth — the circuit remains reserved even during idle periods (e.g., silence in a phone call); setup time is required before transmission can start; not scalable for bursty data traffic like typical Internet usage; if any link in the path fails, the whole circuit fails.

Packet Switching: Data is broken into small units called **packets**, each labelled with source and destination addresses, and packets are transmitted independently through the network, potentially taking different routes, and are reassembled at the destination. There is no dedicated path — the network's bandwidth is shared dynamically among all users. It can be **connectionless (datagram)**, as in IP, where each packet is routed independently, or **connection-oriented (virtual circuit)**, as in older X.25/Frame Relay networks.

- **Benefits:** Efficient utilisation of bandwidth since it is shared among multiple users/sessions dynamically; no dedicated setup time needed (in datagram mode); robust — if one path fails, packets can be rerouted; well suited to bursty, non-continuous data traffic (e.g., web browsing, email).
- **Drawbacks:** Variable delay (jitter) since packets may take different routes and experience different queuing delays; packets may arrive out of order and need resequencing; possibility of packet loss requiring retransmission; more processing/header overhead at each node for routing decisions.

Basis	Circuit Switching	Packet Switching
Path	Dedicated, fixed path reserved for the session	No fixed path — packets may take different routes
Setup	Requires call/circuit setup before data transfer	No setup needed for datagram packet switching
Bandwidth usage	Reserved, can be wasted if idle	Shared dynamically, more efficient
Delay	Constant, predictable delay once set up	Variable delay (queuing at each node)
Order of delivery	In-order (fixed path)	May be out of order; needs resequencing
Failure handling	Whole circuit fails if a link fails	Can reroute around failed links
Example	Traditional telephone network (PSTN)	The Internet (IP-based networks)

Exam tip: If time permits, briefly mention Message Switching as a third (older) technique for a more complete answer.

(b). Compare and contrast POP3 and IMAP. What are the advantages of each protocol for email retrieval?

[CO2]

POP3 (Post Office Protocol version 3) and **IMAP (Internet Message Access Protocol)** are both application-layer protocols used by an email client to **retrieve** email messages from a mail server (they are used for retrieval only — SMTP is used for sending mail).

POP3 works on a 'download and delete' (or 'download and keep') model. When the client connects, it typically downloads all new messages from the server to the local device, and by default deletes them from the server afterward, so mail is stored primarily on the local machine. POP3 uses TCP port 110 (or 995 for POP3S with SSL/TLS).

IMAP works on a 'server-resident' model. Messages remain stored on the mail server, and the client views/manages them remotely — the client only downloads message content when the user opens it. Any folder structure, read/unread status, and flags are synchronised back to the server. IMAP uses TCP port 143 (or 993 for IMAPS with SSL/TLS).

Basis	POP3	IMAP
Storage of mail	Downloaded to and stored on the local device (usually removed from server)	Remains stored on the server; client only views/caches it
Multi-device access	Poor — mail downloaded to one device is not visible on others	Excellent — same mailbox state (read/unread, folders) is visible across all devices
Offline access	Full offline access since mail is stored locally	Limited to previously cached/synced messages
Server storage usage	Low (mail removed after download)	Higher (all mail retained on server)
Speed / bandwidth	Faster initial connection since only headers or full mail downloaded once	Can be slower per session as it syncs folder structure and flags, but downloads only what is needed
Folder management	Limited — mainly a single inbox on server	Full folder/label management synced with server
Port(s)	110 (POP3), 995 (POP3S)	143 (IMAP), 993 (IMAPS)

Advantages of POP3: Simple and lightweight protocol; ideal when a user accesses email from only one device; allows full offline access to already-downloaded mail; uses less server storage since mail is removed after download.

Advantages of IMAP: Ideal for users who check email from multiple devices (phone, laptop, tablet) since the mailbox stays synchronised everywhere; safer against local device loss/damage since mail is kept on the server; supports richer server-side folder organisation and search.

Exam tip: A quick real-world hook: most modern webmail/mobile apps default to IMAP because people use multiple devices — this is a good closing line.

Question 5 (10 Marks)

(a). Define the four types of delays in a network: processing, queuing, transmission, and propagation delays. How does each delay affect overall network performance?

[CO1]

When a packet travels from a source to a destination through a network, it experiences delay at every node (router/switch) along the path. The **total nodal delay** at a node is the sum of four components: processing delay, queuing delay, transmission delay, and propagation delay.

- **1. Processing Delay (d_{proc}):** The time a router takes to examine a packet's header, check for bit-level errors, and determine which outgoing link to forward it on. It depends on the speed of the router's hardware. This delay is usually small (microseconds) in modern high-speed routers.
- **2. Queuing Delay (d_{queue}):** The time a packet waits in a router's output buffer/queue before it can be transmitted, while other packets ahead of it are being sent. It depends heavily on the current level of network congestion — it can range from zero (empty queue) to very large (during congestion), and can even cause packet loss if the buffer overflows.
- **3. Transmission Delay (d_{trans}):** The time required to push all of a packet's bits onto the transmission link, calculated as packet length (L) divided by link bandwidth/transmission rate (R): $d_{\text{trans}} = L / R$. It depends on the packet size and the link's data rate — larger packets or slower links increase this delay.
- **4. Propagation Delay (d_{prop}):** The time it takes for a single bit to travel from the beginning of the link to the end, calculated as the physical distance (d) divided by the propagation speed of the medium (s): $d_{\text{prop}} = d / s$ (propagation speed is close to the speed of light, e.g., $\sim 2 \times 10^8$ m/s in copper/fibre). It depends only on the physical distance and the medium — not on packet size or bandwidth.

Total nodal delay is therefore: $d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$.

Delay Type	Depends On	Effect on Performance
Processing	Router/switch hardware speed, header inspection complexity	Usually negligible, but poor hardware or complex routing rules increase latency
Queuing	Current traffic load / congestion at the router	Most variable and unpredictable; major cause of jitter and delay during congestion
Transmission	Packet size (L) and link bandwidth (R)	Higher for large packets over low-bandwidth links; reduces effective throughput
Propagation	Physical distance and medium (speed of signal)	Fixed for a given link; dominant delay factor over long-distance links (e.g., satellite)

Overall effect: These four delays accumulate at every hop from source to destination, so end-to-end delay is the sum of nodal delays across all routers on the path. High queuing delay causes congestion-related slowdowns and packet loss; high propagation delay (e.g., over satellite or intercontinental links) sets a hard lower bound on latency regardless of bandwidth; and transmission delay limits how quickly large amounts of data can be pushed onto a link. Network designers try to minimise these delays through faster hardware (processing), better queue management/QoS (queuing), higher bandwidth links (transmission), and shorter or more direct physical/routing paths (propagation) — all of which directly improve throughput and user-perceived performance.

Exam tip: Draw the delay formula $d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$ and label a router diagram with each delay occurring at the correct point — this is a standard diagram examiners look for.

(b). What are the primary functions of the DNS? How does it resolve domain names to IP addresses?

[CO2]

DNS (Domain Name System) is a hierarchical, distributed naming system used on the Internet to translate human-readable domain names (e.g., `www.example.com`) into machine-readable numerical IP addresses (e.g., `93.184.216.34`) that computers use to identify each other on the network. It functions much like a 'phone book' for the Internet.

Primary functions of DNS:

- **Name-to-IP resolution:** Translates domain names into IP addresses so that browsers/applications can locate and connect to the correct server.
- **Reverse resolution:** Can also translate an IP address back into its corresponding domain name (PTR records).
- **Host aliasing:** Allows a host to have one or more alias names in addition to its canonical (real) name, using CNAME records.
- **Mail server identification:** Uses MX (Mail Exchange) records to identify which mail servers should receive email for a given domain.
- **Load distribution:** Can distribute load among multiple replicated servers by returning different IP addresses (from a set) for the same domain name in a round-robin fashion.
- **Hierarchical namespace management:** Maintains a distributed, hierarchical database so that no single server needs to store the mapping for every domain in the world, improving scalability and fault tolerance.

DNS Hierarchy: DNS is organised as an inverted tree with three main levels of servers:

- **Root DNS servers** — at the top of the hierarchy; they know the addresses of the TLD servers.
- **Top-Level Domain (TLD) servers** — responsible for domains like `.com`, `.org`, `.net`, `.in`; they know the addresses of the authoritative servers for domains under them.
- **Authoritative DNS servers** — hold the actual DNS records (A, MX, CNAME, etc.) for specific domains and provide the final, authoritative answer.

How DNS resolves a domain name to an IP address (step by step):

1. The user's application (e.g., a browser) needs the IP address for a domain, say `www.example.com`, and sends a query to a **local/recursive DNS resolver** (usually provided by the ISP or a public resolver such as `8.8.8.8`).
2. The resolver first checks its **cache** — if it has recently resolved this domain, it returns the cached IP address immediately, skipping the remaining steps.
3. If not cached, the resolver queries a **root DNS server**, which does not know the exact IP but responds with the address of the appropriate **TLD server** (e.g., for `'.com'`).
4. The resolver then queries the **TLD server**, which responds with the address of the **authoritative DNS server** for `'example.com'`.
5. The resolver queries the **authoritative server**, which holds the actual DNS 'A' record and returns the correct IP address for `www.example.com`.
6. The resolver **caches** this result (for future queries, until the record's TTL expires) and returns the IP address to the user's application.
7. The application now uses this IP address to establish a direct connection (e.g., a TCP connection) with the destination web server.

This process is called **recursive resolution** (from the point of view of the client, who makes one request and gets a final answer) combined with a series of **iterative queries** made by the resolver to the root, TLD, and authoritative servers in turn.

Exam tip: Draw the resolution chain as: Client → Local Resolver → Root Server → TLD Server → Authoritative Server → (answer flows back). This diagram is the single most expected element in DNS answers.

Prepared as a study aid based on the Mid Term (Odd) Semester Examination, October 2024 — Computer Networks (TMC 102), Graphic Era Hill University. Verify against your course notes/textbook before the final exam.

Graphic Era Hill University

Term Evaluation (Odd) Semester Examination — September 2025

Computer Networks (TMC-102)

Model Answer Key — MCA 1st Semester

This document provides detailed, exam-ready answers for **both sub-questions (a) and (b)** of every question in this term evaluation paper, so you can revise the complete syllabus rather than only the options you might choose in the exam. Each answer includes definitions, diagrams described in words, comparison tables, and examples to help you write full 10-mark answers.

Course	MCA — Semester I
Paper	Computer Networks (TMC-102)
Max Marks	50 (5 Questions × 10 Marks)
Instructions followed	Each Q has two options (a)/(b); both are answered here for complete coverage

Question 1 (10 Marks)

(a). Describe the OSI reference model. Explain the functions of each layer in detail.

[CO1]

The **OSI (Open Systems Interconnection) Reference Model**, developed by ISO, is a conceptual 7-layer framework that standardises how different computer systems communicate over a network. Each layer performs a specific set of functions and communicates only with the layer directly above and below it, and with the corresponding (peer) layer on the remote system. Data passed down from the Application layer at the sender is progressively wrapped with header information at each layer (encapsulation), and the reverse (decapsulation) happens at the receiver.

Layer 1 — Physical Layer: Concerned with the transmission of raw, unstructured bits over a physical medium. It defines electrical/optical signal characteristics, voltage levels, cable/connector specifications, data rates, and bit synchronisation. It deals with topology and transmission mode (simplex, half-duplex, full-duplex). Devices: hubs, repeaters, cables.

Layer 2 — Data Link Layer: Responsible for node-to-node (hop-by-hop) delivery of data frames over a single link. Its main functions are framing (dividing the bit stream into frames), physical/MAC addressing, flow control, error detection/correction (using checksums or CRC), and access control (deciding which device gets to use a shared medium, e.g., CSMA/CD). Devices: switches, bridges, NICs.

Layer 3 — Network Layer: Responsible for the delivery of packets from source to destination across multiple networks (end-to-end, host-to-host). Key functions include logical addressing (IP addressing), routing (determining the best path via routing algorithms/tables), and fragmentation/reassembly of packets to suit different network MTU sizes. Devices: routers. Key protocol: IP.

Layer 4 — Transport Layer: Responsible for end-to-end (process-to-process) delivery of the entire message and for ensuring that data is delivered error-free, in sequence, and without loss or duplication. Functions include segmentation and reassembly, connection control (connection-oriented like TCP or connectionless like UDP), flow control, and error control. Uses port numbers to identify specific application processes.

Layer 5 — Session Layer: Establishes, manages, and terminates sessions (dialogues) between two communicating applications. Functions include dialogue control (deciding whose turn it is to transmit — half-duplex/full-duplex dialogue), and synchronisation (inserting checkpoints into a long data stream so that, in case of a crash, only the data after the last checkpoint needs to be resent).

Layer 6 — Presentation Layer: Concerned with the syntax and semantics of the information exchanged — i.e., ensuring the data is in a usable format for the receiving application. Functions include translation (converting between different data representations, e.g., character encoding), encryption/decryption for security, and compression to reduce the number of bits transmitted.

Layer 7 — Application Layer: The topmost layer, closest to the end user, providing network services directly to user applications. Functions include file transfer and access, email services, directory services, and general network virtual terminal support. Example protocols: HTTP, FTP, SMTP, DNS.

Layer No.	Layer Name	Key Function	PDU / Devices
7	Application	Provides network services directly to end-user applications	Data / HTTP, FTP, SMTP
6	Presentation	Data translation, encryption, compression	Data / Gateways

Layer No.	Layer Name	Key Function	PDU / Devices
5	Session	Session establishment, management, synchronisation	Data / Gateways
4	Transport	End-to-end, process-to-process delivery; flow & error control	Segment / TCP, UDP
3	Network	Logical addressing and routing across networks	Packet / Routers
2	Data Link	Framing, MAC addressing, hop-to-hop error/flow control	Frame / Switches, Bridges
1	Physical	Transmission of raw bits over the physical medium	Bit / Hubs, Cables, NICs

Exam tip: Draw the classic 7-layer vertical stack diagram with data flowing down at the sender and up at the receiver, labelling the PDU name (bit/frame/packet/segment/data) at each layer — this is the single most expected diagram for OSI questions.

(b). Define the following types of delays with examples: (i) Processing delay (ii) Queuing delay (iii) Transmission delay (iv) Propagation delay

[CO2]

When a packet travels from source to destination, it experiences delay at each router/node along the path. The total nodal delay is made up of four components:

(i) Processing Delay: The time a router takes to examine a packet's header, check for bit errors, and decide the outgoing link. *Example:* A router takes 2 microseconds to read a packet's destination address and look it up in its forwarding table before sending it onward. It depends on router hardware speed and is usually very small in modern routers.

(ii) Queuing Delay: The time a packet spends waiting in the output buffer of a router before it can be transmitted, because other packets ahead of it are being sent first. *Example:* During a network traffic spike (e.g., many users streaming video at once), a packet may wait several milliseconds — or even get dropped if the buffer is full — before it is its turn to be transmitted. It depends on the level of congestion and is the most variable/unpredictable of the four delays.

(iii) Transmission Delay: The time required to push all of a packet's bits onto the transmission link, given by $d_{trans} = L / R$, where L is the packet length in bits and R is the link's transmission rate (bandwidth) in bits per second. *Example:* Sending a 1000-byte (8000-bit) packet over a 1 Mbps link takes $8000/1,000,000 = 8$ milliseconds. It depends on packet size and link bandwidth.

(iv) Propagation Delay: The time it takes for a single bit to physically travel from the sender's end of the link to the receiver's end, given by $d_{prop} = d / s$, where d is the physical distance and s is the propagation speed of the signal in the medium (close to the speed of light, roughly 2×10^8 m/s in copper or fibre). *Example:* A signal travelling 2000 km through a fibre-optic cable at 2×10^8 m/s takes $2,000,000 / (2 \times 10^8) = 10$ milliseconds to reach the other end. It depends only on physical distance and the medium — not on packet size or bandwidth, which is why satellite links (very large distance) have high propagation delay even with high bandwidth.

Delay Type	Formula / Basis	Typical Cause
Processing	Depends on router CPU speed	Header inspection, error checking, routing lookup
Queuing	Depends on traffic/congestion at the node	Packets waiting in buffer during heavy traffic
Transmission	$d_{trans} = L / R$ (packet size / bandwidth)	Pushing all bits of the packet onto the link
Propagation	$d_{prop} = d / s$ (distance / signal speed)	Physical distance the signal must travel

Total nodal delay = $d_{processing}$ + $d_{queuing}$ + $d_{transmission}$ + $d_{propagation}$. These four delays occur at every hop, so total end-to-end delay is the sum across all routers on the path from source to destination.

Exam tip: Sketch a simple diagram: a packet arriving at a router → sitting in a queue → being processed → being transmitted onto the outgoing link → propagating to the next router. Label each delay at the correct point.

Question 2 (10 Marks)

(a). Differentiate between circuit switching and packet switching. Provide one real-life example of each.

[CO1]

Circuit Switching: A dedicated, fixed communication path is established between the sender and receiver before data transfer begins, and this path remains reserved exclusively for that connection for its entire duration. It involves three phases: circuit establishment, data transfer, and circuit teardown.

Packet Switching: Data is broken into small units called packets, each carrying source/destination address information, and transmitted independently through the network. No dedicated path is reserved; bandwidth is shared dynamically among many users, and packets may take different routes to reach the destination, where they are reassembled.

Basis	Circuit Switching	Packet Switching
Path	Dedicated, fixed path reserved for the entire session	No fixed path; packets may take different routes
Setup	Requires connection setup before transmission	No setup required for datagram-style transmission
Bandwidth usage	Reserved bandwidth, can be wasted if idle	Shared dynamically among users — more efficient
Delay behaviour	Constant delay once the circuit is set up	Variable delay due to queuing at each node
Order of arrival	Data arrives in order (fixed path)	Packets may arrive out of order and need resequencing
Failure impact	Failure of any link breaks the whole circuit	Can reroute around a failed link
Real-life example	Traditional landline telephone call (PSTN)	Browsing the Internet / sending data over IP networks (e.g., WhatsApp messages)

Exam tip: Since the question explicitly asks for a real-life example, always name one clearly for each — e.g., PSTN telephone call for circuit switching, and Internet/IP-based communication (web browsing, email) for packet switching.

(b). Explain the principles of network applications and the client-server architecture.

[CO2]

Network applications are software programs that run on multiple end systems (hosts) connected by a network and communicate with each other to accomplish a task, without requiring any special software to be installed inside the network core (routers/switches) — this is the fundamental design principle of the Internet, often called the **end-to-end principle**. Examples include web browsing, email, video conferencing, file sharing, and online gaming.

Key principles of network application architecture:

- **Process communication:** Communication actually happens between processes (running programs) on different end systems, not directly between the hosts themselves. The process that initiates communication is the **client**, and the process that waits to be contacted is the **server**.

- **Sockets:** A process sends and receives messages through a software interface called a **socket**, which acts as the door between the application layer and the transport layer (network).

- **Addressing:** To send a message to a specific process on another host, the sending process needs the host's IP address and a **port number** that identifies the specific receiving process/application on that host.

- **Transport service selection:** An application chooses the transport protocol that meets its needs — reliable, ordered delivery (TCP) for applications like file transfer/email, or fast, low-overhead delivery (UDP) for applications like live video/voice where occasional loss is tolerable.

Client-Server Architecture is one of the two major network application architectures (the other being peer-to-peer). In this model, there is an always-on **server** host that has a fixed, well-known IP address, and services requests from many different **client** hosts. Clients do not communicate directly with each other — all communication is routed through the server.

- **Working:** The client initiates contact with the server by sending a request; the server, which is always running and listening for connections, processes the request and returns an appropriate response. A single powerful server (or a data centre of many servers, in the case of large services like Google or Netflix) can serve a huge number of clients simultaneously.

- **Characteristics:** Centralised control and easier management/security; server must be scaled (often via data centres and load balancing) as the number of clients grows; single point of failure at the server.

- **Examples:** Web browsing (browser as client, web server as server), email retrieval (mail client, mail server), and cloud storage services.

Exam tip: Contrast briefly with P2P architecture (no dedicated server, every peer can act as client and server) to show a complete understanding, since this is a commonly linked concept.

Question 3 (10 Marks)

(a). Explain SMTP, POP3, and IMAP protocols with their roles in email communication.

[CO2]

Electronic mail (e-mail) systems rely on three main types of protocols working together: one for **sending/relaying** mail (SMTP), and two for **retrieving** mail from a mailbox server (POP3 and IMAP).

SMTP (Simple Mail Transfer Protocol): SMTP is the standard protocol used to **send** email from a client to a mail server, and to relay mail between mail servers, using TCP port 25 (587 for authenticated client submission). SMTP is a **push protocol** — the sending machine pushes the message to the receiving machine. It uses simple text-based commands such as HELO/EHLO (identify sender), MAIL FROM (specify sender address), RCPT TO (specify recipient address), DATA (begin message content), and QUIT. A key limitation is that SMTP cannot be used by a client to pull/retrieve messages from a mailbox — it can only push messages towards a server, which is why separate retrieval protocols (POP3/IMAP) are needed.

POP3 (Post Office Protocol version 3): POP3 is a **pull protocol** used by an email client to retrieve messages from a mail server, over TCP port 110. It typically works on a 'download and delete' model — messages are downloaded to the client's local device and, by default, removed from the server, so mail is primarily stored locally. It is simple and lightweight but does not synchronise mailbox state well across multiple devices, since once mail is downloaded to one device, it is generally no longer visible on the server for other devices.

IMAP (Internet Message Access Protocol): IMAP is also a pull protocol for retrieving mail, over TCP port 143, but it follows a 'server-resident' model — messages remain stored on the mail server, and the client only views and manages them remotely. Actions such as reading, deleting, moving to folders, or flagging a message are synchronised back to the server, so the same mailbox state is visible identically from any device the user logs in from (phone, laptop, webmail).

Protocol	Role	Direction	Port	Mail Storage
SMTP	Sends mail from client to server, and relays between servers	Push	25 / 587	N/A (relay only)
POP3	Retrieves mail; typically downloads and removes from server	Pull	110	Local device
IMAP	Retrieves mail; keeps it synchronised on the server	Pull	143	Server (syncd to client)

Putting it together: When user A sends an email to user B, the sender's mail client uses **SMTP** to push the message to the sender's mail server, which then uses SMTP again to relay it (possibly via intermediate servers) to the recipient's mail server, where it is stored in B's mailbox. User B's mail client then uses **POP3** or **IMAP** to retrieve/access that mail from the mailbox server.

Exam tip: A simple flow diagram — Sender's client --SMTP--> Sender's server --SMTP--> Receiver's server <--POP3/IMAP-- Receiver's client — is exactly what examiners look for in this question.

(b). Describe the TCP/IP protocol stack and compare it with the OSI model.

[CO1]

The **TCP/IP protocol stack** (also called the Internet Protocol Suite) is the practical set of communication protocols that actually implement and run the Internet. Unlike OSI, it was developed based on real, working protocols rather than a theoretical layering scheme. It is generally described using 4 layers:

- **Application Layer:** Combines the functions of OSI's Application, Presentation, and Session layers. It supports protocols that directly serve user applications, such as HTTP (web), FTP (file transfer), SMTP/POP3/IMAP (email), and DNS (name resolution).
- **Transport Layer:** Provides end-to-end (process-to-process) data delivery, corresponding closely to OSI's Transport layer. Uses **TCP** (Transmission Control Protocol) for reliable, connection-oriented, ordered delivery, and **UDP** (User Datagram Protocol) for fast, connectionless, best-effort delivery.
- **Internet Layer:** Corresponds to OSI's Network layer. Responsible for logical addressing (IP addresses) and routing of packets across interconnected networks. Main protocol: **IP** (IPv4/IPv6); also includes supporting protocols like ICMP and ARP.
- **Network Access (Link) Layer:** Combines the functions of OSI's Data Link and Physical layers — handles framing, physical/MAC addressing, and the actual transmission of bits over the physical medium (Ethernet, Wi-Fi, etc.).

Basis	OSI Model	TCP/IP Model
Number of layers	7 layers	4 layers
Development	Layers defined first, protocols fitted later (theoretical)	Protocols developed first, model described afterwards (practical)
Usage	Used as a reference/teaching model	Actually implemented and used to run the Internet
Layer merging	Session, Presentation, Application kept separate	Session, Presentation, Application merged into one Application layer
Protocol dependency	Generic, protocol-independent model	Tied to specific protocols: IP, TCP, UDP, HTTP, etc.
Reliability approach	Reliability can be built into multiple layers	Reliability mainly handled at the Transport layer (TCP)

Exam tip: A layer-mapping diagram (OSI's 7 boxes on one side, TCP/IP's 4 boxes on the other, connected by arrows showing which OSI layers merge into which TCP/IP layer) is the standard diagram expected here.

Question 4 (10 Marks)

(a). Explain HTTP message formats and the role of cookies and proxy servers.

[CO2]

HTTP (HyperText Transfer Protocol) is an application-layer, text-based, request-response protocol used between web browsers (clients) and web servers.

HTTP Request message format consists of: a **request line** (method, URL, HTTP version — e.g., 'GET /index.html HTTP/1.1'), followed by one or more **header lines** (e.g., Host, User-Agent, Accept, Cookie, Connection), a **blank line** marking the end of headers, and an optional **body** (used with methods like POST/PUT to carry form data or JSON).

HTTP Response message format consists of: a **status line** (HTTP version, status code, reason phrase — e.g., 'HTTP/1.1 200 OK'), followed by **header lines** (e.g., Content-Type, Content-Length, Date, Server, Set-Cookie), a **blank line**, and the **body** containing the actual requested resource (HTML, image, JSON, etc.).

Role of Cookies: Since basic HTTP is **stateless** (each request is treated independently, with no memory of previous requests), **cookies** are used to let a server maintain state about a user across multiple requests/visits. When a server first responds to a client, it can include a 'Set-Cookie' header containing a small piece of data (often a unique identifier). The browser stores this cookie and automatically includes it in the 'Cookie' header of every subsequent request to that same server. This allows the server to recognise the returning user and maintain sessions such as shopping carts, login sessions, and personalised preferences.

Role of Proxy Servers: A **proxy server** is an intermediary server that sits between the client and the actual origin (web) server, forwarding client requests and server responses on their behalf. Key roles include:

- **Caching:** Storing (caching) copies of frequently requested web pages/objects so that future requests for the same content can be served directly from the proxy, reducing response time and network traffic.
- **Access control / security:** Filtering requests to block access to certain websites or content, often used in organisations and schools.
- **Anonymity:** Hiding the client's real IP address from the origin server, since the origin server only sees the proxy's address.
- **Load reduction:** Reducing the load on the origin server and the traffic on the access link to the wider Internet, by serving many local users' repeated requests from cache.

Exam tip: Write out a sample GET request with a Cookie header and the matching 200 OK response with a Set-Cookie header — a concrete example scores well here.

(b). Explain DNS resource records and how DNS messages are exchanged between client and server.

[CO1]

DNS (Domain Name System) is a distributed, hierarchical database that maps human-readable domain names to IP addresses and other information, stored across many DNS servers worldwide in the form of **resource records (RRs)**.

DNS Resource Record format: Each resource record is a 4-tuple: **(Name, Value, Type, TTL)**, where TTL (Time To Live) determines how long the record may be cached before it should be refreshed. The

meaning of Name and Value depends on the Type:

Type	Meaning of Name	Meaning of Value	Example Use
A	A hostname	The IPv4 address for that hostname	example.com → 93.184.216.34
AAAA	A hostname	The IPv6 address for that hostname	IPv6 equivalent of an A record
NS	A domain (e.g., example.com)	Hostname of an authoritative DNS server for that domain	Delegates a domain to its name servers
CNAME	An alias hostname	The canonical (real) hostname it points to	ftp.example.com → example.com
MX	A domain	Hostname of the mail server for that domain	Directs email for a domain to its mail server
PTR	An IP address	The hostname corresponding to that IP	Used for reverse DNS lookups

DNS Message Format: DNS queries and replies use the same message format, consisting of: a **header section** (identification number, flags indicating query/reply and whether the answer is authoritative, and counts of the records in each following section), a **question section** (the name being queried and the type of query), an **answer section** (the resource record(s) actually answering the query), an **authority section** (records about other authoritative servers), and an **additional section** (other helpful records, e.g., the IP address of a mail server named in an MX record).

How DNS messages are exchanged between client and server:

- 1. The client's application invokes the **DNS resolver** (a piece of the operating system's networking stack), which constructs a DNS **query message** and sends it, typically encapsulated in a single UDP datagram, to a configured local/recursive DNS server.
- 2. If the local DNS server does not have the answer cached, it forwards the query (as an iterative query) up the hierarchy — first to a **root server**, which replies with a referral to the correct **TLD server**.
- 3. The local server then queries the **TLD server**, which replies with a referral to the **authoritative server** for the specific domain.
- 4. The local server queries the **authoritative server**, which sends back a DNS **reply message** containing the actual resource record (e.g., an A record with the IP address).
- 5. The local DNS server relays this reply back to the original client, and also **caches** the record (for the duration of its TTL) to speed up future queries for the same name.

DNS typically uses **UDP** for these query/reply exchanges (since messages are small and the overhead of setting up a TCP connection is usually unnecessary), though it can fall back to TCP for larger messages such as zone transfers between servers.

Exam tip: Include the resolution-chain diagram (Client → Local DNS → Root → TLD → Authoritative, with the answer flowing back) alongside the RR table for a complete answer.

Question 5 (10 Marks)

(a). Discuss Internet access networks and compare broadband, fiber, and wireless connections.

[CO2]

An **Internet access network** is the physical link that connects an end user's device (home, office, or mobile) to the edge router of their Internet Service Provider (ISP), forming the 'last mile' of the network path to the wider Internet.

1. Broadband (DSL / Cable): Broadband access typically refers to **DSL (Digital Subscriber Line)**, which uses existing telephone copper lines, and **Cable Internet**, which uses the coaxial cable infrastructure originally built for cable television. DSL uses a splitter to simultaneously carry telephone and data signals over different frequency bands on the same copper wire to a DSLAM at the telephone company's local office. Cable networks typically use a shared coaxial/fibre hybrid (HFC) architecture, where bandwidth on the local cable segment is shared among neighbourhood users. Broadband is widely available, relatively affordable, but offers moderate speeds (typically tens of Mbps) that can degrade with distance (DSL) or peak-time congestion from shared neighbours (cable).

2. Fiber (FTTH — Fiber to the Home): Fiber-optic access networks use optical fibre cables to carry data as pulses of light directly (or very close) to the user's premises, often using a Passive Optical Network (PON) architecture where a single fibre from the ISP is split to serve multiple homes. Fiber offers very high bandwidth (hundreds of Mbps to several Gbps), very low latency, and highly symmetric upload/download speeds, with performance that does not degrade with distance nearly as much as copper-based DSL. Its main drawback is that it requires significant infrastructure investment and is not yet available everywhere.

3. Wireless Access (Wi-Fi and Cellular): Wireless access networks connect end devices to the network via radio signals rather than a fixed cable. This includes **Wi-Fi (WLAN)**, used within homes and offices, where devices connect to a local wireless access point which is itself typically connected to a wired broadband/fiber connection, and **cellular/mobile broadband (4G/5G)**, provided by mobile network operators, allowing wide-area, mobile access to the Internet directly through cell towers. Wireless access offers great mobility and flexibility, and modern 5G/Wi-Fi 6 can achieve very high speeds, but performance can be affected by signal interference, distance from the access point/tower, and the number of users sharing the same wireless channel.

Basis	Broadband (DSL/Cable)	Fiber (FTTH)	Wireless (Wi-Fi/Cellular)
Medium	Copper telephone wire / coaxial cable	Optical fibre (light pulses)	Radio waves (no physical cable to device)
Typical speed	Tens of Mbps	Hundreds of Mbps to multi-Gbps	Variable — depends on standard (Wi-Fi 6, 5G) and signal conditions
Latency	Moderate	Very low	Can be higher / more variable
Mobility	None (fixed line)	None (fixed line)	High — supports mobile/roaming use
Reliability	Can degrade with distance/congestion	Very stable, less distance-sensitive	Affected by interference, obstacles, and shared channel congestion

Basis	Broadband (DSL/Cable)	Fiber (FTTH)	Wireless (Wi-Fi/Cellular)
Availability	Widely available in most areas	Limited to areas with fibre rollout	Wide coverage, especially with cellular networks

Exam tip: Mention that most home Wi-Fi is actually the 'last few metres' on top of an underlying broadband/fiber connection — this shows deeper understanding of how the access technologies combine in practice.

(b). Write a short note on BitTorrent and peer-to-peer file distribution. Explain how file sharing happens in this architecture.

[CO1]

Peer-to-peer (P2P) file distribution is a decentralised model in which a file is shared directly among multiple end-user devices (**peers**) rather than being downloaded from a single centralised server. Every peer can simultaneously act as both a client (downloading pieces of the file it doesn't have) and a server (uploading pieces it already has to other peers), which allows the system to scale efficiently since each new peer adds not just demand but also additional upload capacity to the system.

BitTorrent is one of the most popular and widely used P2P file distribution protocols. Its key components and working are as follows:

- **Torrent file / magnet link:** A user wanting to share a file creates a small **.torrent** file (or magnet link) containing metadata about the file — its name, size, and a cryptographic hash of the file divided into fixed-size **chunks/pieces** — along with the address of a **tracker**.
- **Torrent / Swarm:** The collection of all peers currently participating in sharing pieces of a particular file is called a **torrent** or **swarm**. Peers who have the entire file and continue to share it are called **seeders**; peers still downloading pieces are called **leechers**.
- **Tracker:** A tracker server keeps track of which peers are currently part of the swarm for a given torrent, and provides a newly joining peer with a list of other peers currently participating (some modern BitTorrent implementations use a trackerless, DHT-based approach instead).
- **Chunk-based downloading:** When a new peer joins the swarm, it does not download the file from any single source. Instead, it contacts several other peers and begins downloading different pieces of the file from different peers simultaneously, using the hash values in the torrent file to verify that each downloaded piece is correct/uncorrupted.
- **Piece selection strategy:** A common strategy is 'rarest first', where a peer prioritises downloading the pieces that are currently least available among its neighbouring peers, which helps keep all pieces well-distributed across the swarm and prevents any single piece from becoming a bottleneck.
- **Tit-for-tat incentive mechanism:** To discourage 'free-riding' (peers who only download without uploading), BitTorrent uses a 'tit-for-tat' approach — a peer prioritises uploading pieces to the neighbouring peers that are currently providing it with the best download rate in return, encouraging reciprocal, cooperative sharing.
- **Reassembly:** Once a peer has successfully downloaded all the pieces of the file (verified against their hashes), it reassembles them in the correct order to recreate the complete original file, and can then continue acting as a seeder, uploading pieces to other peers still downloading.

This architecture makes BitTorrent highly **scalable** — as more peers join to download a popular file, the total available upload bandwidth in the swarm also grows, unlike a traditional client-server download where a single server's bandwidth is quickly overwhelmed by many simultaneous downloaders.

Exam tip: Draw a swarm diagram: one peer connected to several other peers, with different coloured 'pieces' being exchanged in different directions, plus a tracker box at the side pointing peers toward each other — this visual

captures the whole mechanism clearly.

Prepared as a study aid based on the Term Evaluation (Odd) Semester Examination, September 2025 — Computer Networks (TMC-102), Graphic Era Hill University. Verify against your course notes/textbook before the final exam.